心屿（Isle）

开发全流程与 AI 协同记录（合订本）

AI创新应用挑战赛

开发与 AI 协同合订本

沧州交通学院 · 计算机与信息技术学院 · 软件工程 2410 · 吴思彤

2026 年 10 月 30 日

合订说明

本合订本由三份独立文档合并而成，便于评审连续阅读：

- 第一部分：开发全流程记录（阶段、决策、问题与验证）
- 第二部分：AI 协同开发记录（迭代与关键 Prompt 链）
- 第三部分：AI 对话全记录（按幕整理的用户 Prompt / AI 回复 / 修复过程）

三份文档保留各自原有结构与标题；Git 历史说明、提交材料对照见仓库 README。

合订本目录

1. 第一部分 · 开发全流程记录
2. 第二部分 · AI 协同开发记录
3. 第三部分 · AI 对话全记录

第一部分 · 开发全流程记录

> 本文档记录本项目从 0 到 1 的**真实开发链路**：每一次技术决策、每一个阶段的产出、 以及全程踩到的坑与修复方式。所有条目均可在演示中重演验证。

目录

- 阶段零 · 技术选型与架构决策
- 阶段一 · 工程骨架
- 阶段二 · 数据层与角色系统
- 阶段三 · 端侧语义记忆引擎
- 阶段四 · 视觉体系重构
- 阶段五 · 收尾与交付
- 问题总览
- 附：AI 协同方式说明

阶段零 · 技术选型与架构决策

目标

确定平台范围、技术栈与代码组织方式，并把它固化为对后续所有 AI 生成代码的约束。

真实决策过程

平台范围：赛题为移动端应用。团队资源有限，集中在 Android 单端打磨质量，不做跨平台折中。

技术栈选择依据

维度	评估要点	结论
端侧 AI 推理	核心差异化能力是**端侧语义向量**（长期记忆检索），需要直接调用 ONNX Runtime 原生接口	原生 Kotlin 集成链路最短
系统能力	图片选择、相机、文件存储、输入法、通知等系统 API 直连	原生 Kotlin 无需插件桥接
UI 表达	液态玻璃（真实背景模糊）需要底层渲染控制能力	Compose + 原生渲染可控
构建可控性	需在无 Android Studio 的命令行环境下完成全部构建与验证	Gradle Wrapper + 命令行即可

最终决策：Kotlin + Jetpack Compose 原生 Android。 该决策使端侧向量推理从"跨语言桥接调试"降为"直接调用 API"，是项目能在一个短周期内 跑通核心能力的前提。

架构分层（先定约束，再让 AI 生成代码）

```
┌──────────────────────────────────────────────────────┐
│ UI 层（Jetpack Compose）                              │
│  Screen（无状态）  ←→  ViewModel（StateFlow 单向数据流）│
├──────────────────────────────────────────────────────┤
│ Domain 层                                            │
│  PromptBuilder   —— 角色卡 + 记忆 + 用户资料 → 提示词   │
│  MemoryExtractor —— 对话 → 结构化记忆（LLM）           │
│  BertTokenizer / TextEmbedder / VectorMath           │
├──────────────────────────────────────────────────────┤
│ Data 层                                              │
│  Repository：Character / Chat / Memory               │
│  本地：Room（3 表）+ DataStore（2 组配置）              │
│  远端：OpenAiClient（OpenAI 兼容，SSE）+ ContentFactory │
├──────────────────────────────────────────────────────┤
│ 基础设施                                              │
│  AppContainer（手写容器）· AvatarStorage（图片分级压缩）  │
└──────────────────────────────────────────────────────┘
```

约束原则（写进每次生成 Prompt 的固定前缀）：

1. AI 生成的任何代码必须落在上述四层之内，**禁止跨层直接调用**；
2. 依赖注入使用手写 `AppContainer`，**不引入 DI 框架**（依赖少、结构透明，减少 AI 幻觉面）；
3. 状态一律用 `StateFlow` 从 ViewModel 单向流向 UI，禁止 UI 反向持有可变状态；
4. 高风险能力（端侧模型、网络）必须先定义接口与失败降级路径。

阶段一 · 工程骨架

目标

产出可编译、可安装、可启动的最小工程，为后续所有功能提供稳定的地基。

产出清单

文件	作用
`settings.gradle.kts`	仓库与模块声明、插件仓库配置
`build.gradle.kts`（根）	插件声明（不在此处配置模块）
`gradle/libs.versions.toml`	**版本目录**：所有依赖版本的唯一来源
`gradle.properties`	构建开关（含后续的关键约束）
`gradle/wrapper/*`	Gradle 9.6.0 Wrapper（随仓库分发）
`app/build.gradle.kts`	模块配置：SDK 版本、Compose、签名、密钥注入
`AndroidManifest.xml`	应用声明、FileProvider、软键盘模式

文件	作用
res/values/*	主题、颜色、字符串

关键问题与修复

问题 1：AGP 9 内置 Kotlin 与独立 Kotlin 插件冲突

```
Cannot add extension with name 'kotlin', as there is an extension already registered with that name.
```

- **定位**：反编译 AGP jar，发现 BooleanOption.BUILT_IN_KOTLIN（对应 android.builtInKotlin），确认 AGP 9.4.1 已内置 Kotlin 支持并注册了 kotlin 扩展。
- **首轮修复**：移除独立 org.jetbrains.kotlin.android 插件。
- **连带问题**：移除后 Compose 编译报错 Starting in Kotlin 2.0, the Compose Compiler Gradle plugin is required，补回 org.jetbrains.kotlin.plugin.compose 后通过。

问题 2：Gradle 拉取依赖时 TLS 握手失败

- **现象**：Remote host terminated the handshake
- **定位**：用 curl 单独测试各仓库连通性，均返回 200 → 判定为瞬时抖动而非配置问题。
- **修复**：重试即通过（后续构建未复现）。

验证

```
./gradlew assembleDebug
BUILD SUCCESSFUL in 33s
产物：app-debug.apk（11.6 MB）
```

阶段二 · 数据层与角色系统

目标

打通"角色 → 会话 → 记忆"的数据闭环，并让 AI 能真正回复。

产出清单

Room 数据层 – CharacterEntity：名称、关系类型、性格、背景、说话风格、开场白、头像路径 – MessageEntity：角色 ID 外键、role、内容、图片路径、时间戳 – MemoryEntity：内容、类别、重要度、**embedding 二进制列**、创建/最近使用时间 – 三个 DAO + IsleDatabase（**外键 CASCADE**：删除角色自动清除会话与记忆） – MemoryEntity 手写 equals/hashCode（正确处理 ByteArray 字段）

网络层 – OpenAiApi：Retrofit 接口，使用 @Url 以支持**运行时配置任意 OpenAI 兼容地址** – OpenAiClient：基于 okhttp-sse，支持流式回调与**主动取消**（页面退出即断开） – ContentFactory：构造纯文本 / **多模态（文本 + 图片 base64 内联）** 消息 – DTO 兼容 OpenAI 协议，并忽略推理模型的 reasoning_content 字段

Domain 与仓储 – `PromptBuilder`：把角色卡 + 召回记忆 + 用户资料组装成系统提示词 – `CharacterRepository / ChatRepository / MemoryRepository` – `AppContainer`：集中构建数据库、网络、仓储、工具

UI 与交互 – 角色列表、角色编辑（关系类型选择 + 头像上传）、聊天页、设置页 – `ImagePicker`：相册（Photo Picker，无需权限）+ 拍照（FileProvider） – `AvatarStorage`：图片导入与压缩

关键问题与修复

问题 3：KSP 注入源码被 AGP 9 拒绝

```
Using kotlin.sourceSets DSL to add Kotlin sources is not allowed with built-in Kotlin.
Solution: ... set android.disallowKotlinSourceSets=false in gradle.properties.
```

- **原因**：Room 依赖 KSP 生成代码，而 KSP 需要把生成目录注入 `kotlin.sourceSets`。
- **修复**：按官方提示加入 `android.disallowKotlinSourceSets=false`。（该开关只在内置 Kotlin 方案下需要；升级到独立 Kotlin 插件后已从 `gradle.properties` 移除，当前构建不再依赖它。）

问题 4：依赖的 Kotlin 元数据版本高于编译器

```
Class 'kotlin.reflect.KClass' was compiled with an incompatible version of Kotlin.
The actual metadata version is 2.4.0, but the compiler version 2.2.0 can read versions up to
2.3.0.
```

- **定位**：执行 `dependencyInsight --dependency kotlin-stdlib --configuration debugCompileClasspath`， 显示 **Coil 3.6.3 通过 constraint 强制 `kotlin-stdlib:2.4.10`**； 其余候选版本最高为 2.3.20（2.2.0 编译器可读）。
- **修复**：逐版本核对 POM，确认 Coil **3.4.0** 依赖 stdlib 2.3.10 → 降级至 3.4.0。
- **教训**：这类"版本天花板"问题在阶段四再次出现，最终以升级编译器根治（见问题 10）。

问题 5：聊天图片点开模糊

- **现象**：图片消息点开全屏查看时明显发虚。
- **定位**：沿调用链回溯 `ChatViewModel.attachImage` → `AvatarStorage.importFromUri`， 发现聊天图片**复用了头像的压缩逻辑（512px 上限）**。
- **修复**：为存储逻辑抽出 `maxSize` 参数——头像 512px、**聊天图 1600px**； 同时把大图查看器的容器从 `fillMaxWidth` 改为 `fillMaxSize`。
- **残留说明**：修复前已存入的旧图不追溯，重新发送即清晰。

验证

项目	结果
真机安装启动	正常，无崩溃
角色人设	回复符合设定（闺蜜口吻、使用预设口癖「嗯嗯」「其实呀」）
开场白	首次进入聊天页自动发送角色开场白

项目	结果
图片多模态	发送注册页截图后，AI 准确描述了页面内容（用户名、密码框、性别单选框、按钮、底部提示）

阶段三 · 端侧语义记忆引擎

目标

实现**完全离线**的长期记忆检索——这是本产品区别于同类方案的核心。

产出清单

端侧模型 – `bge-small-zh-v1.5` ONNX **int8 量化版**（23.9 MB）+ `vocab.txt`（21128 词条），随包分发于 `app/src/main/assets/embedding/` – 选型理由：int8 量化在体积（24MB）与精度之间取得平衡；512 维足以支撑短文本记忆检索

推理链路 – `BertTokenizer`：自实现 BERT WordPiece 分词 – CJK 字符逐字切分 → 标点切分 → 贪心最长匹配 → 补 `[CLS]`/`[SEP]` – `TextEmbedder`：ONNX Runtime 推理 – 输入 `input_ids` / `attention_mask` / `token_type_ids` – 输出 `last_hidden_state` → **CLS 池化** → **L2 归一化** → 512 维向量 – `VectorMath`：余弦相似度 + `FloatArray` ↔ `ByteArray`（Little-Endian 序列化）

业务链路 – `EmbeddingProvider` 接口 + `OnnxEmbeddingProvider` 实现（**加载失败自动降级**） – `MemoryRepository`： – 写入：内容去重 → 生成向量 → 落库 – 召回：候选池（按重要度 + 最近使用，限 200 条）→ 加权打分 语义 `0.8` + 关键词 `0.2` + 重要度 `0.1` → Top-K – `MemoryExtractor`：用 LLM 从对话抽取事实，输出 JSON 数组（限 5 条/轮）

界面 – 记忆管理页：分类标签（信息/喜好/经历/关系/约定）、单条删除、一键清空

关键设计：抽象先行 + 降级路径

先实现「关键词 + 重要度」召回，把检索抽象为 `EmbeddingProvider` 接口，再接 ONNX 实现。 这样即使模型加载失败（设备兼容性、内存不足），应用**对话、角色、记忆书写**等能力全部不受影响， 仅召回退化为关键词模式。

关键问题与修复

问题 6：记忆措辞过于生冷

- **反馈原文**：记忆里不要是用户什么什么，太生冷了，要心与心
- **定位**：抽取提示词要求"使用第三人称「用户」"，导致输出如「用户喜欢吃东西。」
- **修复**：重写抽取约束—— `1.` 用「你」称呼对方，写成一句温柔自然的话，像是你在心里默默记住的 `2.` 绝对不能出现「用户」这两个字，也不要写成冷冰冰的资料条目 `3.` 已知的信息不要重复记录，只记这次新出现的内容
- **验证**：真机重新抽取输出—— 记得七月二十号是你的生日，是夏天里海边最舒服的时候、 知道你周末喜欢去爬山，身上总带着一股很清爽的劲儿

问题 7：多角色之间是否记忆串台

- **反馈原文**：多角色不要记忆错乱

- **应对思路**：这类问题**不能靠肉眼确认**，必须用测试固化。
- **做法**：编写仪器化测试 `MemoryIsolationTest` 1. 创建两个角色各写一条记忆 2. 断言各自只能看到自己的记忆 3. 删除其中一个角色 → 断言其记忆被级联清除 4. 断言另一个角色的记忆不受影响
- **结果**：`2 tests, 0 failures`

验证（真实测试数据）

项目	结果
`./gradlew connectedDebugAndroidTest`	2 tests, 0 failures
端侧语义区分度	`sim(爬山, 登山) = 0.8496689` vs `sim(爬山, 股票) = 0.38915947`
向量维度 / 耗时	512 维；3 次推理约 0.4s（真机）
端到端	对话后自动生成记忆并在记忆页可见

阶段四 · 视觉体系重构

目标

从"功能可用"提升到"耐看且统一"，并补齐关键交互缺失。

关键定位：为什么"UI 不统一"

反馈原文：还是不好看啊，而且 UI 没有全局统一

- **排查**：主题开启了 **Material You 动态取色**（`dynamicColor = true`），系统按壁纸生成一套配色，与品牌紫渐变**同时存在**，导致同一界面出现两套色彩体系。
- **修复**：关闭动态取色，改为完整定义的品牌色板（亮/暗双套），并新建 `DesignSystem.kt` 固化圆角体系（10/14/18/22/28）与间距体系（4/8/12/16/20/24）。

产出清单

项	内容
主题	关闭动态取色；定义 primary/secondary/tertiary + 容器色 + 描边色的完整色板
设计规范	`DesignSystem.kt`：`IsleShapes` / `IsleSpacing` / `IsleRadius`
液态玻璃	接入 **Haze 2.0.1**，实现真实背景模糊（backdrop blur）+ 磨砂噪点
玻璃组件	`GlassCard`：半透明渐变填充 + 白色高光描边 + 柔和彩色投影
首页重构	去掉悬浮加号按钮 → 列表内「+ 创建新角色」卡片；新增 Hero 卡与分组标题
交互补全	聊天页右上角由「设置」改为「编辑角色」；图片全屏查看（双指缩放 + 拖动）
键盘适配	`windowSoftInputMode="adjustResize"` + 各页 `imePadding()`

项	内容
合规	隐私文案由"仅保存在本机"改为准确表述（说明会传输至模型服务）

关键问题与修复

问题 8：应用内键盘唤不起来

- **反馈原文**：有个严重问题点说点什么拉不起来输入法！！！
- **排查（用系统状态做裁判）**： `bash adb shell dumpsys input_method # mInputShown=true # mServedView=AndroidComposeView # mServedInputConnection=NullableInputConnectionWrapperApi34` 系统明确表示"输入法已显示且已连接"，说明 **App 侧请求链路正常**； 进一步临时切换系统键盘 → 键盘立刻弹出 → **判定为设备输入法配置问题**（该机默认输入法被设成了微信输入法的插件服务）。
- **App 侧仍做加固**：`adjustResize` + `imePadding()`，确保键盘弹出时输入框被顶到上方。

问题 9：液态玻璃效果不达标

- **反馈原文**：不够透膜砂 → 之后又出现 感觉首页上面特别模糊
- **过程**：先调强（`blurRadius` 48dp + `noiseFactor` 0.14 + 低不透明度）， 首页大面积高饱和渐变卡滚入顶栏后被糊成色雾； 最终收敛为 `blurRadius 22dp + noiseFactor 0.06 + 底色 alpha 0.14`。
- **教训**：玻璃效果的强度必须与"背后内容的复杂度"匹配，不能一味调参。

问题 10：Haze 2.x 无法使用（版本天花板的根治）

```
Module was compiled with an incompatible version of Kotlin.
The binary version of its metadata is 2.4.0, expected version is 2.2.0.
```

- **第一次尝试（错误方向）**：降级 Haze 到 1.7.3 —— 能编译，但只能使用旧 API，效果受限。
- **关键转折**：提出"能否升级 Kotlin? "，随后系统评估升级路径： 1. 查 KSP 版本列表 → 最新为 `2.3.12`，**没有 2.4 系列**，若升到 Kotlin 2.4 会导致 Room 注解处理不可用； 2. 编译器版本 N 可读取 metadata 至 N+1 → **Kotlin 2.3.x 恰好可读 metadata 2.4.0**； 3. 结论：升到 **Kotlin 2.3.21**，既解锁 Haze 2.x，又保持 KSP 可用。
- **升级后遇到新阻碍**： ⛔ `The 'org.jetbrains.kotlin.android' plugin is not compatible with AGP's 9.0 new DSL (`android.newDsl=true` is enabled by default). Solution: ... set `android.newDsl=false` in gradle.properties` 按官方提示同时设置 `android.builtInKotlin=false` + `android.newDsl=false`，改用独立 Kotlin 插件。
- **API 不确定性处理**：不猜测写法，直接反编译依赖 jar 取精确签名—— `bash javap -classpath . dev.chrisbanes.haze.blur.HazeBlurKt # hazeBlur(Modifier, HazeInput, HazeBlurStyle, HazePerformanceMode, boolean)` 依据真实签名生成调用代码，**一次编译通过**。

问题 11：批量去重命令误删源码

- **现象**：使用 `awk '!seen[$0]++'` 去重导入语句时，该命令会删除文件中**所有**重复行 （包括合法的重复代码行），导致 `CharacterListScreen.kt` 结构损坏、编译报错。
- **修复**：整文件重写恢复。

- **流程改进**：此类"全局去重"手法此后禁止用于源码文件。

验证

真机逐页截图确认：首页、聊天页、设置页、角色编辑页、技术信息页、图片查看器。

阶段五 · 收尾与交付

代码与依赖清理

- 删除死代码：`EmbeddingProvider.NoOp`、`ContentFactory.emptyObject()`、`SettingsRepository.updateMaxTokens`、`AvatarStorage.file()`、`ChatRepository.observe()/clear()`、`MessageDao` 中对应的未用方法
- 移除未使用依赖：`okhttp-logging`、`ui-tooling-preview`、`room-ktx`（Room 2.6+ 已并入 runtime）
- 清理版本目录中未使用的插件别名
- 结果：`compileDebugKotlin --rerun-tasks` **BUILD SUCCESSFUL，Kotlin 0 警告**（Gradle 仍有 2 条 AGP 9 迁移提示）

体积优化

- **问题**：ONNX Runtime 自带四种 CPU 架构的原生库，全量打包导致 APK 达 **185MB**
- **方案**：启用 `splits.abi`，仅保留 ARM 架构并拆分产物
- **结果**：**185MB → 67MB（arm64）/ 58MB（armv7）**

文档交付

- `README.md`：10 分钟极速复现指南
- `docs/开发与AI协同全记录.md`：本合订本（开发全流程与问题记录）
- `docs/技术文档.md`：意图控制策略 / 架构设计 / 可复现性论证
- 本合订本第二部分：AI 协同开发记录（关键 Prompt 链还原）
- 本合订本第三部分：AI 对话全记录（脱敏；完整记录在本地数据库）

Git 演进记录

为便于归档，10–16 将 10–15 的开发成果按模块整理为 12 个基础提交 （骨架 → 主题 → 数据层 → 网络层 → 配置 → 提示词与仓储 → 记忆引擎 → 界面 → 玻璃视觉 → 测试 → 文档），随后按真实时间继续提交文档与修复。 **不存在一次性 `initial commit`；但前 12 个提交的时间戳是整理时间，而非每一步的实时时间。**

交付后修复（2026–10–17，不计入前述 13 个开发期问题）

问题	修复
冷启动时年龄门会闪一下	区分「配置尚未加载」与「未确认年龄」，加载完成前不渲染年龄门
编辑角色后角色跳到列表末尾	`CharacterRepository.save` 更新时保留原 `createdAt`

问题	修复
用户/助手消息同毫秒插入时排序可能错位	`ORDER BY timestamp DESC, id DESC`；助手消息 id 直接取插入返回值
SSE 的 `[DONE]` 与 `onClosed` 可能各触发一次	`AtomicBoolean` 保证只结算一次，避免重复抽取记忆
ONNX 输入张量未释放，长对话泄漏 native 内存	`OnnxTensor.use { ... }` 显式关闭
「仅保存在本机」与系统云备份矛盾	`android:allowBackup="false"`，不参与云备份/换机迁移
聊天页附图弹窗仍写「选择头像来源」	图片选择器支持标题参数，聊天页传「发送图片」
记忆解析只看首个 `[` 与末个 `]`	真正实现 `removeThinkTags()` + `extractJSON()`，先清除思考块/代码围栏

阶段六 · 交付前收口（2026-10-26～10-29）

> 交付后新增的一轮收口，目标是让演示更稳定、日常使用更可靠；最终只保留单版本。

- 构建收口：移除双构建，使用主包名，版本保持 `v1.0.0`；
- 演示数据：3 个角色 / 6 条记忆 / 7 条示例对话，设置页可重置；
- 聊天可靠性：停止生成、失败重试、清空会话，错误不写入助手消息；
- 图片链路：采样解码、UUID 文件名、删角色/清空会话清理图片、拍照缓存清理；
- 设置能力：年龄分组重选、最大 tokens、测试连接（地址/模型/Key 可用性）；
- 记忆质量：语义近似去重；
- 数据演进：Room v2 + 复合索引 + Migration(1→2)；
- 视觉：窗口背景与深色主题对齐。

阶段七 · 模型服务通用化（2026-10-29）

- 网络层改为 OpenAI 兼容协议，移除 DeepSeek 专用客户端；
- `local.properties` 支持 `AI_BASE_URL` / `AI_MODEL` / `AI_API_KEY`，并兼容旧键；
- 设置页加入服务预设与 `/models` 可搜索列表，模型可随时切换；
- 本地 / 局域网 `http://` 服务可用，Key 可留空；
- 模拟器缺少 ARM64 指令时，端侧向量自动降级，避免启动崩溃；
- 验证：33 个 JVM 单测 + 3 个真实 E2E；千问 MaaS `qwen3.5-omni-plus` 非流式/流式均通过。

问题总览

#	阶段	问题	定位手段	修复方式
1	一	AGP 9 内置 Kotlin 与独立插件冲突	反编译 AGP jar 查选项	首轮移除独立插件；升级 Kotlin 2.3.21 后改回独立插件 + `builtInKotlin=false`
2	一	依赖拉取 TLS 握手失败	`curl` 单独验证连通性	判定瞬时抖动，重试
3	二	KSP 注入 sourceSets 被拒绝	读取官方报错提示	`android.disallowKotlinSourceSets=false`（后期已移除）
4	二	Kotlin 元数据版本不兼容	`dependencyInsight` 查依赖树	Coil 降级至 3.4.0
5	二	聊天图片模糊	沿调用链回溯	抽出 `maxSize`，聊天图 1600px
6	三	记忆措辞生冷	用户反馈 + 检查抽取提示词	重写为第一人称温情口吻
7	三	多角色记忆隔离	无法肉眼确认 → 编写测试	`MemoryIsolationTest` 固化
8	四	键盘唤不起来	`dumpsys input_method` 采集状态	判定设备问题；App 侧加 `imePadding`
9	四	玻璃强度失衡	真机截图对比	收敛模糊半径与噪点
10	四	Haze 2.x 元数据不兼容	评估 KSP 生态 → 升级编译器	Kotlin 2.3.21 + 关闭新 DSL
11	四	去重命令误删源码	编译报错定位	重写文件 + 禁用该手法
12	四	UI 不统一	排查主题配置	关闭动态取色 + 建立设计规范
13	五	APK 体积过大	解包统计各目录占比	按 CPU 架构拆分

附：AI 协同方式说明

本项目的开发全程采用 **Vibe Coding** 范式，核心方法是：

1. 先定约束，再生成代码

每次让 AI 生成前，先用自然语言明确"分层边界 + 数据流 + 命名规范 + 验收标准"， 而不是直接说"帮我写个聊天页"。例：

> 写聊天页的 ViewModel。输入是 `characterId`；它从 `AppContainer` 取 `chatRepository` / `memoryRepository` / `openAiClient`；发送流程是： 先落库用户消息 → 再插占位助手消息 → 流式更新 → 完成后落库并异步抽取记忆； 对外暴露 StateFlow；验收标准是编译通过。

2. 用编译器和测试做裁判

AI 给出的方案不靠"看起来对"，一律通过下列手段验证：

手段	用途
`compileDebugKotlin --rerun-tasks`	强制全量编译，杜绝增量缓存的假通过
`connectedDebugAndroidTest`	端侧向量语义区分度、多角色记忆隔离
`dependencyInsight`	定位依赖冲突的真实来源
`dumpsys input_method` / `logcat`	区分"App 问题"与"设备问题"
`javap` 反编译依赖 jar	在 API 不确定时直接从字节码确认方法签名

3. 报错即上下文（闭环排错法）

把**完整证据链**回灌给 AI，而不是只描述现象： 报错原文（完整栈 + 关键行）+ 依赖树 + 环境事实（Gradle / JDK / AGP / Kotlin 版本）+ 已尝试的修复及结果。

4. 抽象层先行

对高风险能力（端侧向量、记忆检索、网络）先定义接口与降级路径，再替换实现， 保证任何一步失败都不会让应用不可用。

> 完整的 Prompt 链与修 Bug 过程见本合订本第二部分与第三部分。

第二部分 · AI 协同开发记录

> **从立项、开发、调试到成品** 一份以「意图 → 方案 → 报错 → 定位 → 修复 → 验证」为主线的 AI 协同实录

项目	内容
项目	心屿（Isle）· 原生 Android AI 情感陪伴应用
协作方式	Vibe Coding：人类定义约束与验收标准，AI 生成与执行，编译器/测试/真机做裁判
时间跨度	2026–10–15 15:51 — 2026–10–17 15:30
核心链路	角色卡 + 全局用户资料 + 端侧语义记忆 + DeepSeek 流式对话
交付物	APK、6 份 Markdown 文档 + 2 份 PDF、17 张截图、40+ 次提交

> **说明** 本文中的用户发言为**保留原意的书面化整理**，不是逐字稿；AI 方案、报错原文、命令输出、测试数据均来自真实执行结果。 完整的 785 条消息（含 749 次工具调用）保存在本地会话数据库，不随包分发。

一页看懂

阶段	时间	目标	结果
立项与选型	10–15 15:51—16:03	产品形态、平台、技术栈	Kotlin + Compose + Room + ONNX
工程骨架	16:03—16:07	可编译、可安装	25 个文件；首个 APK 11.6MB
数据与网络	16:07—16:24	角色 / 会话 / 记忆 + DeepSeek	Room 三表、SSE、图片多模态
端侧记忆	16:25—16:30	完全离线的语义检索	bge–small–zh int8、512 维、2 tests
资料与隐私	16:31—16:55	可交付的设置体系	四个二级页、密钥注入、隐私修正
体验攻坚	16:55—17:18	图片 / 输入法 / 质感	1600px、IME 责任边界、玻璃卡片
视觉重构	17:18—17:42	全局统一	Haze 2.0.1、Kotlin 2.3.21、ABI 拆分
交付与合规	10–16—10–17	文档 / 截图 / 未成年保护	24 页技术文档、17 页说明书、年龄门
交付后修复	10–17 15:00—15:30	真机验收问题	8 项修复；2 tests / 0 failures

迭代零 · 立项与技术选型

目标

在写第一行代码前，把「做什么、给谁做、在哪个平台做、记忆怎么实现」定清楚。

意图（整理）

#	用户诉求	落地结论
1	先熟悉既有 Web 原型，输出架构与关键实现	识别出多 Provider、情感分析、长期记忆缺口
2	评估重构为原生 Android，模型只接 DeepSeek-V4.1-Flash	可行；砍掉多 Provider 与后端代理
3	产品形态为 AI 女友/男友/闺蜜/朋友	定位「可自定义角色 + 长期记忆」
4	角色可自定义性格、简介，并保留记忆	设计 `Character` / `Memory` 两个核心实体
5	DeepSeek 暂无 Embedding API	改为端侧 `bge-small-zh-v1.5` ONNX
6	默认角色 + 多角色 + 用户上传头像；暂不做语音/立绘	收敛 v1 范围，头像落应用内部目录
7	新建 Android 工程，命名「心屿」	包名 `com.isle.companion`，仅 Android

决策

- **平台**：原生 Kotlin + Jetpack Compose。端侧 ONNX 推理需要直连原生接口，跨平台方案的 FFI 成本更高。
- **架构**：UI（Compose）→ Domain（Prompt/Memory）→ Data（Room/DataStore/DeepSeek）→ 手写 `AppContainer`，不引入 DI 框架。
- **记忆**：LLM 抽取事实 → 端侧向量化 → Room BLOB → 本地加权召回，不依赖云端向量库。
- **边界**：不含自建服务端；对话内容只在发送时传输至用户配置的模型服务。

复盘

- 先收敛范围，再写代码。最早一版曾考虑语音、立绘、实时通话，最终全部推迟到 v1 之后。
- 「记忆」不是附加功能，而是产品定义的一部分；它决定了数据模型、提示词与隐私表述。

迭代一 · 工程骨架：先让项目跑起来

目标

产出一个可编译、可安装、可启动的最小工程，为后续所有功能提供稳定地基。

意图（整理）

> 新建一个原生 Android 工程，应用名「心屿」，包名 `com.isle.companion`；使用较新的 Kotlin + Compose 组合，先搭一个能编译、能跑起来的骨架。

方案

生成 `settings.gradle.kts`、根 `build.gradle.kts`、版本目录 `libs.versions.toml`、`gradle.properties`、Manifest、`MainActivity`、Compose 主题与占位首页，并立即执行构建。

报错与定位

报错 1

```
Cannot add extension with name 'kotlin', as there is an extension already registered with that name.
```

定位：反编译 AGP jar 的选项类，发现 `BooleanOption.BUILT_IN_KOTLIN`（对应 `android.builtInKotlin`），确认 AGP 9.4.1 已内置 Kotlin 编译器并注册了 `kotlin` 扩展。

首轮修复：移除独立的 `org.jetbrains.kotlin.android` 插件，改用 AGP 内置 Kotlin。

报错 2（连带问题）

```
Starting in Kotlin 2.0, the Compose Compiler Gradle plugin is required when compose is enabled.
```

修复：只补回 `org.jetbrains.kotlin.plugin.compose`，不重新引入 Kotlin Android 插件。

验证

```
./gradlew assembleDebug
BUILD SUCCESSFUL in 33s
产物：app-debug.apk（11.6 MB）
```

复盘

- **不猜 API**：让 AI 去读依赖字节码，而不是「回忆文档」。
- **连带问题预判**：移除插件前先确认 Compose 编译是否依赖它。
- 这一轮建立的目录结构、版本目录与构建约束，后续 40+ 次提交都沿用。

迭代二 · 数据层与网络层：打通「角色 → 对话」

目标

建立 Room 三表与 DeepSeek 客户端，让应用真正能创建角色、发送消息、收到流式回复。

意图（整理）

> 实现本地数据层：角色、消息、记忆三张表；记忆表预留 `embedding` 二进制列；删除角色时级联清除会话与记忆。 网络层统一为 OpenAI 兼容协议：地址 / 模型 / Key 均可在应用内修改，支持 SSE 流式与主动取消、图片 base64 内联，DTO 兼容 `reasoning_content`；默认接入 DeepSeek，也可切换 OpenAI、通义千问 / 千问 MaaS 或本地服务。

方案

- `CharacterEntity / MessageEntity / MemoryEntity` + 三个 DAO + `IsleDatabase`；

- `DeepSeekClient`（当时的实现，现已统一为 `OpenAiClient`）：Retrofit 非流式 + OkHttp SSE 流式；
- `ContentFactory`：纯文本 / 文本 + 图片多模态；
- `PromptBuilder`：角色卡 + 用户资料 + 召回记忆 + 最近 12 条历史。

报错与定位

报错 3（KSP 注入源码被拒）

```
Using kotlin.sourceSets DSL to add Kotlin sources is not allowed with built-in Kotlin.
Solution: ... set android.disallowKotlinSourceSets=false in gradle.properties.
```

处理：按官方提示加入 `android.disallowKotlinSourceSets=false`。

> 该开关只在内置 Kotlin 方案下需要；升级 Kotlin 2.3.21、改用独立插件后已移除。

报错 4（依赖元数据版本不兼容）

```
Class 'kotlin.reflect.KClass' was compiled with an incompatible version of Kotlin.
The actual metadata version is 2.4.0, but the compiler version 2.2.0 can read versions up to
2.3.0.
```

定位

```
./gradlew :app:dependencyInsight \
  --dependency org.jetbrains.kotlin:kotlin-stdlib \
  --configuration debugCompileClasspath
```

```
org.jetbrains.kotlin:kotlin-stdlib:2.4.10
  Selection reasons:
    - By constraint
+--- io.coil-kt.coil3:coil-android:3.6.3
```

→ **Coil 3.6.3 通过 constraint 强制 stdlib 2.4.10**。逐版本核对 POM 后确认 Coil **3.4.0** 依赖 stdlib 2.3.10，降级后构建通过。

验证

项目	结果
安装启动	正常，无崩溃
角色人设	回复符合设定（闺蜜口吻、使用「嗯嗯」「其实呀」）
开场白	首次进入聊天页自动发送
图片多模态	发送截图后，AI 准确描述页面内容
APK	21.5MB；解包确认新类已进包

复盘

- **报错即上下文**：回灌 `dependencyInsight` 的完整输出，而不是只贴报错行。
- **降级是临时方案**：同一类元数据问题在视觉阶段再次出现，最终通过升级 Kotlin 根治。

迭代三 · 端侧语义记忆：产品真正的差异化

目标

让角色「记得你」，且记忆检索完全在手机本地完成，不依赖云端向量库。

意图（整理）

> 记忆检索使用端侧中文 embedding，模型放 assets 随包分发。 先定义抽象接口与降级路径：模型加载失败时，对话、角色、记忆书写都不能受影响。 分词器自行实现，不引入重型 NLP 依赖。

方案

```
EmbeddingProvider（接口）
  └─ OnnxEmbeddingProvider（加载失败返回 null）

BertTokenizer      —— WordPiece（CJK 逐字切分 + 贪心最长匹配）
TextEmbedder       —— ONNX Runtime，CLS 池化 + L2 归一化 → 512 维
VectorMath         —— 余弦相似度 + FloatArray ↔ ByteArray
MemoryRepository   —— 写入补向量；召回 语义0.8 + 关键词0.2 + 重要度0.1
MemoryExtractor    —— LLM 抽取结构化记忆（JSON）
```

关键问题 1：记忆措辞过于生冷

意图（整理）

> 记忆文案不要再用「用户……」这种第三人称冷叙述，改成角色第一人称、有温度的表述。

定位：抽取提示词要求使用第三人称「用户」，输出如「用户喜欢吃东西。」

修复后的抽取约束

```
1. 用「你」称呼对方，写成一句温柔自然的话，像是你在心里默默记住的
2. 绝对不能出现「用户」这两个字，也不要写成冷冰冰的资料条目
3. 已知的信息不要重复记录，只记这次新出现的内容
4. 每条独立完整，能脱离上下文理解
5. 只记明确出现的信息，不要推测
```

验证输出

```
记得七月二十号是你的生日，是夏天里海边最舒服的时候
知道你周末喜欢去爬山，身上总带着一股很清爽的劲儿
```

关键问题 2：多角色记忆是否串台

意图（整理）

> 多角色记忆必须严格隔离；这类问题不能靠肉眼确认，需要用测试固化。

做法：`MemoryIsolationTest` 四步断言—— 创建两个角色各写一条记忆 → 断言互不可见 → 删除其一 → 断言级联清除且不影响另一个。

验证（真机）

```
./gradlew connectedDebugAndroidTest
→ 2 tests, 0 failures

[EmbedderTest] sim(爬山, 登山) = 0.8496689
[EmbedderTest] sim(爬山, 股票) = 0.38915947
```

语义相近句 **0.85**，无关句 **0.39**；512 维，3 次推理约 0.4s（真机）。

复盘

- **抽象先行**：先定义接口与失败回退，再替换实现。
- **不可见的正确性 → 测试固化**：记忆隔离这种「看不见」的约束必须写成断言。
- **情感需求也是需求**：措辞温度通过提示词工程直接解决。

迭代四 · 设置、资料与隐私：从「能用」到「可交付」

目标

建立可交付的设置体系：全局用户资料、模型配置、密钥注入、准确的隐私表述。

意图（整理）

> 设置页需要项目介绍与开发背景；API Key 放到二级页面；评估演示 Key 打入 APK 的风险。 增加全局「我的资料」，让所有角色都能快速了解用户；同时确保隐私文案与真实数据流一致。

方案

模块	实现
设置主页	关于心屿内嵌 + 我的资料 / 模型服务 / 回复风格 / 技术亮点四个入口
模型服务	二级页配置 API Key、地址、模型 ID；演示 Key 构建时注入
我的资料	称呼 / 性格 / 自述，DataStore 全局共享
隐私文案	从「仅保存在本机」改为「本地存储，发送时传输至模型服务」

密钥方案

```
# local.properties（不入库）
DEEPSEEK_API_KEY=sk-...
```

构建时注入 `BuildConfig.DEMO_DEEPSEEK_API_KEY`；优先级：**用户填写的 Key > 内置演示 Key**。

> 演示 APK 中的密钥可被反编译提取，只用于演示；正式发布需改为自建后端代理。

关键解释：为什么资料不能「只发一次」

用户问：既然有知识库，为什么资料不能只发送一次就长期生效?

回答：对话接口是无状态的——模型不会跨请求保留上下文。所谓「记住」，是应用在每次请求时把 **角色卡 + 召回记忆 + 用户资料**重新拼进系统提示词。资料只占几十个 token，成本可忽略； 真正需要控制的是历史窗口与记忆条数。

验证

- 真机确认设置页四个入口与两处隐私文案；
- 「我的资料」进入系统提示词与记忆抽取提示词；
- 记忆抽取增加「已知信息不重复记录」规则。

复盘

- **安全要说清代价**：密钥注入不是「藏起来」，而是「演示可接受、正式发布不可接受」。
- **隐私文案必须与数据流一致**：一句「仅保存在本机」如果被理解成「不出设备」，就是误导。
- **无状态 API 需要应用层「记忆」**：这解释了为什么记忆库是产品核心，而不是附属功能。

迭代五 · 体验缺陷攻坚：用证据链定位问题

5.1 图片点开模糊

意图（整理）

> 聊天中发出的图片点开看不清，需要支持全屏查看并保证清晰度。

定位路径

```
ChatScreen（点击图片）→ ChatViewModel.attachImage
  → AvatarStorage.importFromUri → scaleDown(bitmap, 512)   ← 问题在此
```

根因：聊天图片复用了头像的 512px 压缩逻辑。

修复：抽出 `maxSize` 参数——头像 512px、聊天图 **1600px**；查看器由 `fillMaxWidth` 改为 `fillMaxSize`。

5.2 输入法唤不起来

意图（整理）

> 聊天页点击输入框无法唤起输入法，这是严重阻断问题，请定位根因。

排查（用系统状态做裁判）

```
adb shell dumpsys input_method | grep -E "mInputShown|mServedView|mCurMethodId"
# mInputShown=true
# mServedView=androidx.compose.ui.platform.AndroidComposeView
# mServedInputConnection=NullableInputConnectionWrapperApi34
```

系统明确表示「输入法已显示且已连接」→ App 请求链路正常。 临时切换系统键盘 → 键盘立刻弹出 → 判定为**设备输入法配置问题**（默认输入法被设为微信输入法的插件服务）。

App 侧加固：`windowSoftInputMode="adjustResize"` + 各页 `imePadding()`。

5.3 UI 质感偏弱

意图（整理）

> 当前界面偏「素」，卡片与顶栏缺少层次；希望在视觉上更有质感，并保持全局统一。

处理：先补玻璃卡片、渐变与阴影；随后发现真正的「玻璃」需要真实背景模糊，进入迭代六。

复盘

- **沿调用链回溯**：图片模糊不是渲染问题，而是上游压缩参数问题。
- **用系统状态区分责任边界**：`dumpsys` 一锤定音，避免在 App 侧做无效修复。
- **先可用，再好看**：视觉问题拆成「层次 → 统一 → 真实模糊」三步。

迭代六 · 液态玻璃与视觉体系

目标

从「功能可用」走到「视觉统一」：真实背景模糊、设计规范、包体控制。

意图（整理）

> 目标视觉是 iOS 那种液态玻璃：通透、磨砂、有真实背景模糊；同时保持 9 个页面风格一致。 右下角悬浮加号不美观，首页移除编辑角色入口。

关键问题 1：UI 不统一的根因

排查发现主题开启了 **Material You 动态取色**（`dynamicColor = true`），系统按壁纸生成一套配色，与品牌紫渐变同时存在 → 同一界面出现两套色彩体系。

修复：关闭动态取色；新建 `DesignSystem.kt` 固化圆角（10/14/18/22/28）与间距（4/8/12/16/20/24）。

关键问题 2：Haze 2.x 元数据不兼容

报错

```
Module was compiled with an incompatible version of Kotlin.
The binary version of its metadata is 2.4.0, expected version is 2.2.0.
```

第一次尝试（方向错误）：降级 Haze 到 1.7.3，能编译但只能使用旧 API，效果受限。

转折：用户提出「能否升级 Kotlin？」——随后评估升级路径：

1. 查 KSP 版本列表 → 最新 `2.3.12`，**没有 2.4 系列**；升到 Kotlin 2.4 会导致 Room 注解处理不可用；
2. 关键事实：**Kotlin 编译器版本 N 可读取 metadata 至 N+1**；
3. 结论：升到 **Kotlin 2.3.21**——既解锁 Haze 2.x，又保持 KSP 可用。

升级后的新阻碍

```
The 'org.jetbrains.kotlin.android' plugin is not compatible with AGP's 9.0 new DSL
(`android.newDsl=true` is enabled by default).
Solution: ... set `android.newDsl=false` in `gradle.properties`.
```

修复

```
android.builtInKotlin=false
android.newDsl=false
```

并应用独立 Kotlin 2.3.21 + KSP 2.3.12。

关键问题 3：Haze API 不确定

意图（整理）

> 不要猜 API 写法。直接反编译 jar，把 `hazeBlur` 的精确签名与相关包名查出来。

```
javap -classpath . dev.chrisbanes.haze.blur.HazeBlurKt
# public static final androidx.compose.ui.Modifier hazeBlur(
#     Modifier, HazeInput, HazeBlurStyle, HazePerformanceMode, boolean)

unzip -l haze-blur-materials-2.0.1.aar | grep HazeMaterials
# → dev/chrisbanes/haze/blur/materials/HazeMaterials.class
```

依据真实签名生成调用代码，一次编译通过。

关键问题 4：玻璃强度失衡

参数	调整前	调整后
`blurRadius`	48dp	**22dp**
`noiseFactor`	0.14	**0.06**
玻璃底色 alpha	0.10	**0.14**

教训：玻璃强度必须与「背后内容的复杂度」匹配，不能一味调参。

包体优化

```
dex              64.5 MB
lib/x86_64       39.1 MB
lib/x86          39.0 MB
lib/arm64-v8a    32.8 MB
lib/armeabi-v7a  23.2 MB
assets           23.0 MB
```

根因：ONNX Runtime 自带四套 CPU 架构原生库。 **修复**：`splits.abi` 仅保留 ARM → **185MB 降至 67MB（arm64）/ 58MB（armv7）**。

一次真实失误

使用 `awk '!seen[$0]++'` 去重导入语句时，该命令会删除文件中所有重复行（包括合法代码行），导致 `CharacterListScreen.kt` 损坏。

修复：整文件重写。 **流程改进**：此类「全局去重」手法此后禁止用于源码文件。

复盘

- **同一类问题第二次出现时改变策略**：从「逐个降级依赖」升级为「评估并升级编译器」。
- **升级前先评估生态约束**：KSP 无 2.4 → 选择「编译器 2.3.x + 可读 metadata 2.4.0」的临界版本。
- **官方报错自带解法**：AGP 的报错直接给出了 `android.newDsl=false`。

迭代七 · 从功能到交付

7.1 文档、截图与仓库

意图（整理）

> 按交付要求梳理完整开发流程，只写真实发生的过程；产出 README、技术文档、使用说明书与 AI 记录，并用真机截图替换占位图。

产出

文档	结果
README	10 分钟复现指南（环境、构建、安装、测试、排错）
技术文档	24 页；意图控制 / 架构 / 端侧记忆 / 提示词 / 可复现性
使用说明书	17 页；9 张真机截图
开发日志	五阶段产出 + 13 个真实问题
AI 记录	协同 Prompt 链 + 对话节选
仓库	Public；基础提交 12 个，后续按真实时间继续提交

关键决策：PDF 用 Markdown → HTML → Chrome Headless 生成；截图全部来自真机，不使用设计稿冒充。

7.2 未成年保护：从「告知」到「分级限制」

第一版：选择「恋人 / 自定义」时弹出免责声明。 **问题**：新建角色默认关系就是「恋人」，用户不点选项直接保存即可绕过。

修复：默认关系改为中性的「闺蜜」；只有主动选择亲密关系才触发声明。

第二版：用户指出「仅靠弹窗无法阻止未成年人使用」。 **方案**：年龄门 + 青少年模式——

机制	落地
年龄门	首次启动确认年龄段；未确认不进入主界面
青少年模式	隐藏「恋人」「自定义」；设置页显示标识
内容约束	系统提示词追加最高优先级保护条款
记忆约束	不记录涉及亲密关系的内容
能力边界	年龄为自声明；实名核验需服务端与合规资质

原有点击弹窗与对应截图一并移除；文档、截图、PDF 同步更新。

7.3 成品状态

维度	状态
应用	心屿 v1.0.0；`minSdk 26` / `compileSdk 37`
角色	预置小屿 / 阿岸 / 林深 + 多角色自定义（性格/背景/说话风格/头像）
对话	DeepSeek SSE 流式；图片多模态；可停止、失败重试、清空会话
记忆	LLM 抽取 + 端侧 512 维向量 + 本地加权召回 Top-6
合规	年龄门 + 青少年模式 + 提示词/记忆双重约束
演示数据	3 个角色 / 6 条记忆 / 7 条示例对话，可重置

维度	状态
质量	Kotlin 0 警告；2 tests / 0 failures；arm64 APK 68MB

迭代八 · 交付后修复

> 交付验收后，又在真机上发现 8 个问题。它们不改变产品形态，但直接影响「能不能放心交付」。

#	问题	修复
1	冷启动时年龄门闪一下	区分「配置未加载 / 未确认」，加载完成前只渲染背景
2	编辑角色后列表顺序变化	更新时保留原 `createdAt`
3	同毫秒消息排序可能错位	`ORDER BY timestamp DESC, id DESC`；助手 id 取插入返回值
4	SSE 完成回调可能重复结算	`AtomicBoolean` 只允许结算一次
5	ONNX 输入张量未释放	`OnnxTensor.use { }` 显式关闭
6	「仅存本机」与云备份矛盾	`allowBackup=false`
7	聊天页附图弹窗文案错误	图片选择器支持标题，聊天页显示「发送图片」
8	记忆解析只看首尾方括号	实现 `removeThinkTags()` + `extractJSON()`

验证

```
./gradlew compileDebugKotlin --offline --rerun-tasks
BUILD SUCCESSFUL

./gradlew connectedDebugAndroidTest
2 tests, 0 failures
```

真机复测：年龄门、编辑角色顺序、消息顺序、图片弹窗、联网流式回复、记忆抽取与召回均通过。

复盘

- **交付不是终点**：真正的问题往往在「装机后、联网后、反复操作后」才出现。
- **每个修复都要有可验证的判据**：数据库顺序、返回 id、单次结算、manifest 标志。
- **文档要与代码同时收口**：页数、隐私表述、构建约束、提交历史说明一并校准。

附录 · 开发期问题索引（13 个）

#	阶段	问题	定位与修复
1	骨架	AGP 9 内置 Kotlin 与独立插件冲突	反编译 AGP jar 查选项；首轮移除独立插件，后期升级 Kotlin 后改回独立插件 + `builtInKotlin=false`
2	骨架	Maven 依赖 TLS 握手失败	`curl` 验证四源均 200，判定瞬时抖动，重试通过
3	数据	KSP 注入 sourceSets 被拒	官方报错给出的 `disallowKotlinSourceSets=false`（后期移除）
4	数据	依赖元数据 2.4.0 高于编译器 2.2.0	`dependencyInsight` 定位 Coil 3.6.3，降级到 3.4.0
5	体验	聊天图片点开模糊	沿调用链回溯，聊天图复用头像 512px 压缩；改为 1600px
6	记忆	记忆措辞「用户……」过于生冷	重写抽取提示词为第一人称、禁止「用户」二字
7	记忆	多角色记忆是否串台	无法肉眼确认，编写 `MemoryIsolationTest` 固化
8	体验	应用内键盘唤不起来	`dumpsys input_method` 判定设备输入法问题；App 侧补 `imePadding`
9	视觉	玻璃强度失衡	真机截图对比，`blurRadius` 48→22dp、`noiseFactor` 0.14→0.06
10	视觉	Haze 2.x 元数据不兼容	评估 KSP 生态后升级 Kotlin 2.3.21，并关闭 AGP 新 DSL
11	视觉	去重命令误删源码	`awk` 删除合法重复行；整文件重写，禁用该手法
12	视觉	UI 不统一	发现 Material You 动态取色；关闭并建立设计规范
13	交付	APK 体积 185MB	解包统计四套 ABI 原生库；`splits.abi` 拆分到 67/58MB

> 另有交付后修复 8 项与交付前收口 1 轮，见「迭代八」「迭代九」。

迭代九 · 交付前收口（10-26～10-29）

> 按交付要求收口为**单版本**：保留主包名 `com.isle.companion`、版本 `v1.0.0`。

改造	结果
构建收口	只保留 `assembleDebug` 一个构建；移除双构建配置
演示数据	首次启动写入 3 个角色、6 条记忆、7 条示例对话；设置页可一键重置
聊天可靠性	新增停止生成、失败重试、清空会话；错误不再写入助手消息
图片链路	先读尺寸再采样解码；文件名 UUID；删角色/清空会话清理聊天图；拍照缓存定期清理
设置能力	年龄分组重选、最大 tokens、测试连接/余额、演示数据重置
记忆质量	语义近似去重（余弦 ⩾ 0.92 合并），减少近义重复
数据演进	Room v2：`messages(characterId, timestamp)` 复合索引 + Migration(1→2)
视觉	窗口背景对齐主题，新增 `values-night`，减少冷启动白闪

验证：`assembleDebug` 通过；演示数据 3/6/7；设置页测试连接返回余额；重置演示数据恢复 3/6/7；年龄分组可回到年龄门；覆盖安装验证过 v1→v2 迁移。

收口结果：单版本，`com.isle.companion`，v1.0.0，arm64 68MB。

迭代十 · 模型服务通用化（10-29）

> 从「只接 DeepSeek」改为「任意 OpenAI 兼容服务」，并让模型可以在设置页直接切换。

改造	结果
协议层	删除 DeepSeek 专用客户端，统一为 `OpenAiClient` + `OpenAiUrls` + SSE/错误解析
配置	`local.properties` 支持 `AI_BASE_URL` / `AI_MODEL` / `AI_API_KEY`，兼容 `OPENAI_*` 与旧 `DEEPSEEK_API_KEY`
运行时	地址 / 模型 / Key 可在设置页随时覆盖；支持本地 / 局域网 `http://`，本地服务 Key 可留空
模型切换	设置页提供 OpenAI / DeepSeek / 通义千问 / 千问 MaaS / 本地 Ollama 预设；`GET /models` 后可在可搜索列表中点选
模拟器兼容	模拟器缺少 ARM64 指令时，端侧向量自动降级为关键词匹配，不再启动崩溃

验证：33 个 JVM 单测 + 3 个真实 E2E（`-Disle.e2e=1`）全部通过；千问 MaaS `qwen3.5-omni-plus` 实测 `/models` 返回 184 个模型，非流式与流式均正常；`scripts/verify-ai-connection.sh` 三步全过。

协同方法总结

1. 先定契约，再生成代码

每次让 AI 生成前，先明确「分层边界 + 数据流 + 命名规范 + 验收标准」。

> **反例**：帮我写个聊天页面。 **本项目写法**：写聊天页 ViewModel。输入 `characterId`；从 `AppContainer` 取仓储与客户端；发送流程是「落库用户消息 → 插占位助手消息 → 流式更新 → 完成后落库并异步抽取记忆」；对外暴露 `StateFlow`；验收标准是编译通过。

2. 用编译器和测试当裁判

手段	用途	实例
`compileDebugKotlin --rerun-tasks`	强制全量编译，杜绝增量缓存假通过	每次改动后必跑
`connectedDebugAndroidTest`	仪器化测试	向量区分度、记忆隔离
`dependencyInsight`	定位依赖冲突真实来源	查出 Coil 强约束 stdlib
`dumpsys input_method`	读取系统真实状态	键盘问题责任边界
`javap` 反编译 jar	确认 API 方法签名	Haze `hazeBlur` 签名
解包统计	定位包体膨胀来源	四套 ABI 原生库

3. 报错即上下文（闭环排错法）

回灌给 AI 的应是完整证据链，而不是现象描述：

```
报错原文（完整栈 + 关键行）
+ 依赖树（谁引入了冲突版本）
+ 环境事实（Gradle / JDK / AGP / Kotlin 版本）
+ 已尝试的修复及结果
```

4. 抽象先行，预留降级路径

高风险能力（端侧向量、网络）先定义接口与失败回退，保证任何一步失败都不会让应用不可用。

5. 同类问题第二次出现时，改变策略

依赖元数据不兼容出现两次：第一次「逐个降级库」，第二次改为**评估并升级编译器**，根治整类问题。

6. 官方报错往往自带解法

AGP 的两处报错都直接给出了配置项（`android.disallowKotlinSourceSets=false`（后期移除）、`android.newDsl=false`）。 **先完整读完报错再动手**，比搜索更快。

验证与数据

指标	结果
开发期真实问题	13 个（全部定位到根因并修复）
交付后修复	8 个 + 交付前收口 1 轮
仪器化测试	2 tests / 0 failures
端侧向量	512 维；相近句 0.85 / 无关句 0.39
记忆召回	语义 0.8 + 关键词 0.2 + 重要度 0.1；Top-6
包体	185MB → 68MB（arm64）/ 58MB（armv7）
全量编译	BUILD SUCCESSFUL；Kotlin 0 警告
提交	40+ 次；10–16 按模块整理 12 个基础提交

结语

这个项目不是「让 AI 一次写对」的演示，而是一份**把失败也纳入流程**的记录：

- 报错不是障碍，是定位的入口；
- 测试不是装饰，是把「看不见的正确性」固定下来；
- 文档不是事后补的说明，而是与代码同步收口的交付物。

从一句「可以重构为原生 Android 吗」，到一款能记住你、能通过真机验收的应用—— 真正可复用的不是某一版代码，而是这套**定义约束 → 生成 → 验证 → 回灌 → 修正**的协作方式。

第三部分 · AI 对话全记录

> **从立项、开发、调试到成品** —— 一份可复核的 AI 协同开发记录
> 项目：心屿（Isle）· 原生 Android AI 情感陪伴应用 记录范围：2026-10-15 15:51 — 2026-10-17 15:30 会话标识：`ses_ec174bc86ffelV9DA2riRKejB6`

阅读说明

- 本文以会话数据库为底本，保留**用户意图、AI 可见回复与关键验证结论**；隐去内部推理与重复工具调用，便于阅读与归档。
- 用户发言在**保留原意**的前提下做了书面化整理；AI 回复、报错原文、命令输出与技术数据均保持原样。
- 全文按「立项 → 骨架 → 核心功能 → 体验攻坚 → 视觉重构 → 交付 → 交付后修复」分幕，每幕附目标、产出与复盘。
- 原始 785 条消息（含 749 次工具调用）保存在本地会话数据库，不随包分发。

全程时间线

阶段	时间	目标	关键产出
立项与选型	10-15 15:51—16:03	产品形态、平台、技术栈	Kotlin + Compose + Room + ONNX；命名「心屿」
工程骨架	16:03—16:07	可编译、可安装	25 个文件；首个 APK 11.6MB
核心闭环	16:07—16:30	角色 / 对话 / 记忆	Room 三表、SSE、端侧向量、2 tests
资料与隐私	16:31—16:55	可交付的设置体系	四个二级页、API Key 注入、隐私文案修正
体验攻坚	16:55—17:18	图片 / 输入法 / 质感	1600px 聊天图、IME 责任边界、玻璃卡片
视觉重构	17:18—17:42	全局统一	Haze 2.0.1、Kotlin 2.3.21、ABI 拆分
交付与合规	10-16—10-17	文档 / 截图 / 未成年保护	README、技术文档、说明书、年龄门
交付后修复	10-17 15:00—15:30	真机验收问题	8 项修复；2 tests / 0 failures

序幕 · 立项：从 Web 原型到原生 Android（10-15 15:51—16:03）

> **本幕目标** 在写第一行代码前，把「做什么、给谁做、在哪个平台做、记忆怎么实现」定清楚。

轮次	用户诉求（整理）	结论
1	先完整熟悉既有 Web 原型，输出架构与关键实现	识别出多 Provider、情感分析、长期记忆缺口

轮次	用户诉求（整理）	结论
2	评估重构为原生 Android，模型只接 DeepSeek-V4.1-Flash	可行；砍掉多 Provider 与后端代理
3	产品形态是 AI 女友/男友/闺蜜/朋友	定位「可自定义角色 + 长期记忆」
4	角色要能自定义性格、简介，并保留记忆	设计 `Character` / `Memory` 两个核心实体
5	DeepSeek 暂无 Embedding API	改为端侧 `bge-small-zh-v1.5` ONNX
6	默认角色 + 多角色 + 用户上传头像；暂不做语音/立绘	收敛 v1 范围，头像落应用内部目录
7	新建 Android 工程，命名「心屿」	包名 `com.isle.companion`，仅 Android

立项结论

- **平台**：原生 Kotlin + Jetpack Compose。端侧 ONNX 推理需要直连原生接口，跨平台方案的 FFI 成本更高。
- **架构**：UI（Compose）→ Domain（Prompt/Memory）→ Data（Room/DataStore/DeepSeek）→ 手写 `AppContainer`，不引入 DI 框架。
- **记忆**：对话后由 LLM 抽取事实 → 端侧向量化 → Room BLOB → 本地加权召回，全程不依赖云端向量库。
- **边界**：不含自建服务端；对话内容仅在发送时传输至用户配置的模型服务。

第一幕 · 工程骨架与核心闭环（10-15 16:03—16:30）

> **本幕目标** 从零搭出可编译、可安装的原生工程，并打通「角色 → 对话 → 记忆」主链路。
>
> **验收标准** `assembleDebug` 通过；真机可创建角色、流式对话、写入并召回记忆。

AI `10-15 16:04`

阶段一文件已建好，现在生成 Gradle Wrapper（用已缓存的 9.6.0，在临时目录生成后拷贝，避免污染构建）。

AI `10-15 16:04`

Wrapper 就绪。现在跑构建验证（首次会联网拉取 AGP/Kotlin/Compose 依赖，耗时较长）。

AI `10-15 16:04`

构建暴露出一个关键点：**AGP 9 已内置 Kotlin 支持**，再单独应用 `org.jetbrains.kotlin.android` 会冲突。我查证一下官方迁移方式。

AI `10-15 16:05`

构建失败原因已定位：**AGP 9.4.1 内置 Kotlin**，与独立 Kotlin 插件冲突。我调整构建脚本：移除独立 Kotlin 插件，改用 AGP 内置 Kotlin 支持。

AI `10-15 16:05`

需要 Compose 编译器插件。重新加入（仅 Compose 插件，不含 Kotlin Android 插件）。

AI 10-15 16:06

进展了：Kotlin/Compose 已配置通过。现在是 Maven 仓库 TLS 握手问题。我测一下各镜像连通性。

AI 10-15 16:06

连通性正常（刚才是瞬时抖动）。重试构建。

用户 · 第 1 轮 10-15 16:07

> 按既定方案开始实现，进入数据层与核心功能开发。

AI 10-15 16:07

开始阶段二。先查所需依赖的最新版本（Room/KSP/Retrofit/OkHttp/Coil/序列化等），并验证 AGP 9 内置 Kotlin 下 KSP 是否可用（Room 的关键前提）。

AI 10-15 16:07

版本已查定。现在进入阶段二，先落 **依赖 + Room 数据层**（这是 KSP 与 AGP 9 内置 Kotlin 兼容性的关键验证点）。

AI 10-15 16:08

数据层文件就绪，先构建验证 KSP（Room 注解处理）能否在 AGP 9 内置 Kotlin 下正常工作。

AI 10-15 16:08

KSP 向 `kotlin.sourceSets` 注入生成代码，与 AGP 9 内置 Kotlin 冲突。按官方提示加抑制开关。

AI 10-15 16:18

好，保留 Kotlin，继续阶段二。先验证刚才的 KSP 兼容开关是否生效（Room 编译通过）。

AI 10-15 16:18

Kotlin 编译器和标准库版本不匹配（编译器 2.2.0 vs 标准库 2.4.10）。先查是谁引入了 2.4.10。

AI 10-15 16:19

Coil 3.6.3 强制拉入了 Kotlin 2.4.10 标准库，而 AGP 内置编译器是 2.2.0，无法读取 2.4 元数据。我查一下 AGP 是否允许指定内置 Kotlin 版本。

AI 10-15 16:19

发现 AGP 提供开关 `android.builtInKotlin`。我打算改用独立 Kotlin 插件（较新版本）以匹配这些 2026 年的库。先确认相关插件版本是否存在。

AI 10-15 16:20

Coil 3.4.0 才用 stdlib 2.3.10（2.2.0 编译器可读）。先降级 Coil 试构建。

🤖 AI **10-15 16:20**

构建通过 ✅ KSP + Room 在 AGP 9 内置 Kotlin 下已跑通（关键：`android.disallowKotlinSourceSets=false` + Coil 降级 3.4.0）。

继续阶段二：网络层 + 设置存储 + 提示词构建。

🤖 AI **10-15 16:21**

现在写提示词构建、仓储层、头像存储与依赖容器。

🤖 AI **10-15 16:21**

先构建验证网络层与容器能否编译。

🤖 AI **10-15 16:21**

网络层通过 ✅ 。现在写 UI 层：ViewModel、角色列表/编辑、聊天、设置与导航。

🤖 AI **10-15 16:23**

构建显示成功，但速度异常快。我核实一下新代码确实被编译进了 APK。

🤖 AI **10-15 16:23**

阶段二完成，构建通过并已核实类文件真实进包。

已完成内容

原生 Android 应用「心屿」已经是一个**可运行的角色陪伴 App**（文字聊天 + 图片输入 + 长期记忆骨架）。

数据层（Room，已生成并编译） – `CharacterEntity / MessageEntity / MemoryEntity` + 三个 DAO + `IsleDatabase` – 角色删除时级联清除消息与记忆；`MemoryEntity` 手写了 `equals/hashCode` 处理 `ByteArray`

网络层（DeepSeek，OpenAI 兼容） – `DeepSeekClient`：非流式（Retrofit + `@Url` 支持运行时改地址）+ **流式**（OkHttp `okhttp-sse`），可取消 – `ContentFactory`：纯文本 / **多模态（图片 base64 内联）** 消息构造 – DTO 含 `reasoning_content` 字段，兼容推理型输出

业务层 – `PromptBuilder`：角色卡（名称/关系/性格/背景/说话风格）+ 长期记忆 → 系统提示词；组装最近 12 条历史 – `CharacterRepository / ChatRepository / MemoryRepository`（记忆召回目前是「关键词 + 重要度」轻量版，留了阶段三接向量的接口） – `AvatarStorage`（相册/拍照图片压缩到 512px 存内部目录）、`AppContainer` 轻量依赖容器

界面（Compose + Material3 + Navigation） – 角色列表（默认角色「小屿」首次启动自动创建）、角色编辑（含关系类型 Chip、头像上传）、聊天页（气泡、流式追加、图片附件、自动滚动）、设置页（API Key/地址/模型/温度） – 图片选择器支持**相册 + 拍照**（FileProvider 已配置） – 主色「岛屿暮紫」，支持深色模式与动态取色

验证

- `./gradlew assembleDebug` → BUILD SUCCESSFUL
- APK：`app/build/outputs/apk/debug/app-debug.apk`（21.5MB），已用 dex 反查确认 `ChatScreen / DeepSeekClient / IsleDatabase` 等类真实打包

- 检测到**已连接真机**（3B15B401PLC00000）

两个必须知道的点

1. **DeepSeek 模型 ID 默认填的是 deepseek-chat**（我无法确认"v4.1 flash"的真实 ID），已在设置页开放修改——请按官方文档填准确值。
2. 构建踩坑已固化：AGP 9 内置 Kotlin 需 `android.disallowKotlinSourceSets=false`，且 **Coil 必须锁 3.4.0**（3.5+ 要求 Kotlin 2.4，与内置编译器 2.2.0 不兼容）。

待你确认

1. **是否安装到已连接的真机试跑？**（`adb install` + 启动，会向你的手机安装应用）
2. `HomeScreen.kt` 已被新界面取代、成为**未使用文件**，是否授权我删除？（按规范删除需你同意）
3. 是否继续**阶段三**（端侧 embedding 记忆检索 + 记忆自动抽取 + 记忆管理界面）？

👤 用户 · 第 2 轮　10-15 16:24

> 模型侧改用 DeepSeek-V4.1-Flash（原生多模态），请同步更新默认模型配置与界面文案。

🤖 AI　10-15 16:24

收到，`deepseek-flash` 即 V4.1 Flash，且**原生多模态视觉**——正好对应我实现的图片输入方案。我把默认模型 ID 改成官方值。

👤 用户 · 第 3 轮　10-15 16:25

> 请依次完成三件事：① 真机安装验证；② 清理无用代码；③ 继续下一阶段。

🤖 AI　10-15 16:25

界面渲染正常（"心屿"、默认角色「小屿·闺蜜」、设置与新建按钮都在）。现在清理无用文件，并开始阶段三的模型调研。

🤖 AI　10-15 16:25

HomeScreen 已清理。`BAAI/bge-small-zh-v1.5` 官方仓库没有 ONNX 导出，我找现成的 ONNX 版本。

🤖 AI　10-15 16:26

`Xenova/bge-small-zh-v1.5` 有 ONNX 导出（含 int8 量化版）。下载模型与词表到 assets。

🤖 AI　10-15 16:26

模型就绪（int8，23.9MB；输入 `input_ids/attention_mask/token_type_ids`，输出 `last_hidden_state`）。查 ONNX Runtime Android 版本。

🤖 AI　10-15 16:26

ONNX Runtime 1.31.0。开始阶段三实现，先加依赖与端侧推理核心（分词器 + 向量工具 + 嵌入器）。

🤖 AI　10-15 16:28

构建通过。为了真实验证端侧 embedding（这是阶段三风险最高的部分），我加一个仪器化测试，在真机上跑一次模型推理。

🤖 AI　10-15 16:29

测试在真机（CPH2747）上通过。取一下具体相似度数值。

🤖 AI　10-15 16:29

端侧 embedding 真机验证通过： – `sim(爬山, 登山) = 0.85`，`sim(爬山, 股票) = 0.39` —— 语义区分明显，3 次推理约 0.4s

顺手清掉构建里的图标弃用告警（统一改用 AutoMirrored 版本）。

🤖 AI　10-15 16:30

APK 涨到 185MB —— 因为 onnxruntime 打包了全部四种 CPU 架构的原生库。先看构成再优化。

🤖 AI　10-15 16:30

主要体积来自 4 种 CPU 架构的原生库（134MB）+ dex（64.5MB）。做 ABI 拆分，只保留 ARM 架构。

本幕小结

维度	产出
工程	Gradle 9.6 + AGP 9.4.1 + Kotlin 2.2.10；首个 APK 11.6MB
数据	Room 三表（角色/消息/记忆）+ 级联删除；`MemoryEntity` 处理 `ByteArray`
网络	DeepSeek 非流式 + SSE 流式（可取消）+ 图片 base64 多模态
端侧记忆	`bge-small-zh-v1.5` int8（23.9MB）+ WordPiece + ONNX 512 维
验证	仪器化测试 2 tests / 0 failures；APK 按 ABI 拆分后 67MB / 58MB

踩坑主线：AGP 9 内置 Kotlin 冲突 → KSP 注入 sourceSets → Coil 3.6.3 拉高 stdlib 2.4.10 → 降级 Coil 3.4.0。

第二幕 · 角色、资料与隐私（10–15 16:31—16:55）

> **本幕目标** 把「能用」推进到「可交付」：设置体系、全局用户资料、密钥注入、隐私表述、记忆语气。
> **验收标准** 设置页四个二级入口可用；资料进入提示词；隐私文案与真实数据流一致。

👤 用户 · 第 4 轮　10-15 16:31

> 优化设置页：补充项目介绍与开发背景，语气可以温暖一些；开发者信息写「软件工程 2410 吴思彤」。

🤖 AI　10-15 16:31

收到。我来做三件事：优化设置页结构、新增「关于心屿」页面（含项目介绍 + 煽情背景 + 开发者信息）、接通导航。先开启 BuildConfig 以便显示版本号。

👤 用户 · 第 5 轮　10-15 16:32

> 把 API Key 配置移到设置页的二级页面，避免主页面堆叠太多表单项。

🤖 AI　10-15 16:32

明白，把 API 配置移到二级页面。设置主页只保留入口，进去再填。

👤 用户 · 第 6 轮　10-15 16:34

> 评估把演示用 API Key 打进 APK 的可行性与风险，并给出方案对比。

🤖 AI　10-15 16:34

可以，但**不安全**，我先说清楚代价，再给可用方案。

直接封进 APK 的现实

APK 是公开可解包的，把 Key 写进 `BuildConfig`、`strings.xml` 或 assets，用 jadx/apktool 或直接 `strings` 就能在几十秒内提取出来。R8 混淆、字符串加密、塞进 native 库只能提高门槛，**挡不住有心人**。

后果：Key 泄露后别人可以用来消耗你的额度/余额（真金白银），也可能违反服务商条款；提交物里还可能出现"硬编码密钥"的负面评价。

四种方案对比

方案	安全	评委体验	成本
A. 用户自己填（现状）	✅ 最好	需手动配置	无
B. **打包演示 Key（构建时注入）**	⚠️ 可提取	✅ 开箱即用	泄露风险
C. 自建后端代理保管 Key	✅ 好	✅ 开箱即用	需服务器
D. 服务器发放短期令牌	✅ 最好	✅	需服务器

我的建议（针对演示）

方案 B 的稳妥做法，兼顾演示和安全： – Key 不写进代码仓库，而是放在 `local.properties`（已在 `.gitignore` 里），构建时注入为 `BuildConfig` 字段 – 优先级：**用户在设置里填的 Key > 内置演示 Key**（用户仍可覆盖） – 设置页状态显示改为「已内置演示密钥」而不是「未配置」 – 明确风险告知：这个 APK 只用于演示，**不要公开分发**，演示后建议到 DeepSeek 控制台把该 Key 停用/轮换

如果这个项目以后要上架或公开分享，就必须走 **方案 C**（自建代理），否则等于把钱包公开。

需要你确认

1. 要哪个方案？（A 保持现状 / **B 构建时注入** / C 需要你有服务器）
2. 若选 B：**请你提供演示 Key**（你可以直接发给我，或告诉我你想自己填到 `local.properties` 里，我只写好注入逻辑）

确认后我再改代码——**在你回复前我不动任何文件**。

👤 用户 · 第 7 轮　10-15 16:35

> 选择方案 B：构建时注入演示密钥。

🤖 AI　10-15 16:35

好，做构建时注入。你没给我 Key，所以我把注入机制写好，**你只需把 Key 填到 `local.properties` 一行里**（该文件已在 `.gitignore` 中，不会进仓库）。

🤖 AI　10-15 16:35

构建验证注入机制是否真正生效（先用临时假值测试，再还原为空）。

🤖 AI　10-15 16:35

`java.util` 在 Kotlin DSL 里被解析成 `java` 扩展了，改用顶部 import。

🤖 AI　10-15 16:35

注入机制验证通过（假值 → 正确写入；清空 → 还原为空）。安装确认界面无异常。

🤖 AI　10-15 16:35

完成。API Key 已支持**构建时注入**，机制已实测验证。

实现方式

环节	说明
密钥来源	`local.properties` 里的 `DEEPSEEK_API_KEY=`（该文件已被 `.gitignore` 忽略，不会进仓库）
注入	`app/build.gradle.kts` 读取后生成 `BuildConfig.DEMO_DEEPSEEK_API_KEY`
优先级	**用户设置的 Key > 内置演示 Key**（设置页填了就用自己的）
状态显示	用内置密钥时，设置页显示「已内置演示密钥 · deepseek-flash」
配置页提示	提示「当前使用应用内置的演示密钥…填写自己的 Key 后将优先使用你的配置」

验证结果

- 临时写入假值 → `BuildConfig.java` 中正确出现该值 ✅
- 清空 → 正确还原为空 ✅

- `./gradlew assembleDebug` BUILD SUCCESSFUL，真机安装启动无崩溃 ✅

你现在只需要做一件事

编辑 `/Users/yna/Desktop/Isle/local.properties`，把 Key 填在这一行等号后面：

```
DEEPSEEK_API_KEY=sk-你的真实密钥
```

然后重新构建：

```
cd /Users/yna/Desktop/Isle && ./gradlew assembleDebug
```

产物：`app/build/outputs/apk/debug/app-arm64-v8a-debug.apk`（自动带上演示密钥，评委开箱即用）

两点必须提醒

1. **这个 APK 里的密钥可被提取**，只用于演示，**不要公开分发**（不要传到网盘/群/仓库）。
2. 演示结束后，建议到 DeepSeek 控制台**停用或轮换**该密钥，避免额度被盗用。若以后要公开上架，必须改成自建后端代理。

另外：我没有向任何地方输出或记录你的密钥，目前项目里也是空的，等你填。

👤 用户 · 第 8 轮　10-15 16:37

> 演示密钥已写入 `local.properties`，请重新构建并做端到端验证。

🤖 AI　10-15 16:37

收到。我先确认 Key 已就位（只显示掩码，不输出明文），然后重新构建。

🤖 AI　10-15 16:37

角色卡片位置已定位（[56,421]–[1216,729]）。进入聊天页。

🤖 AI　10-15 16:38

聊天页已打开，**开场白自动发送成功**（"嗨，你来啦～今天过得怎么样呀？"）。现在发一条消息，实测流式回复。

🤖 AI　10-15 16:38

端到端跑通了！回复完全符合角色设定（闺蜜口吻、用「嗯嗯」「其实呀」）。现在验证**记忆是否真的被抽取入库**。

🤖 AI　10-15 16:38

端到端全链路验证成功。真实测试结果：

实测记录（真机 CPH2747）

1. 开场白自动发送 ✅ 进入聊天页即出现：嗨，你来啦～今天过得怎么样呀？

2. 流式对话 + 人设生效 ✅ 发送消息后（adb 打字被手机输入法自动纠错弄乱了，非 App 问题），角色回复：

> 嗯嗯，我听懂啦～今天又弄到这么晚，肯定累坏了吧。其实呀，再忙也要记得吃点东西垫垫肚子哦…

——完全符合设定的「闺蜜」人设（"嗯嗯""其实呀"正是我写进 `speakingStyle` 的口癖），说明角色卡 → 提示词 → 模型 的链路生效。

3. 记忆自动抽取 + 入库 + 界面展示 ✅ 打开聊天页右上角「长期记忆」，实际看到 4 条自动生成的记忆：| 分类 | 内容 | |---|---| | 喜好 | 用户喜欢吃东西。 | | 经历 | 用户表示今天又很晚，并感到疲惫。 | | 经历 | 用户今天感到疲惫。 | | 经历 | 用户今天又一次忙到很晚。 |

——「对话 → LLM 抽取 → 分类打标 → 写入本地（含端侧向量）→ 记忆管理页展示」全链路打通。

当前状态总结

能力	状态
演示密钥内置	✅ 已注入（sk-ca31****）
流式回复	✅ 真机验证
角色人设	✅ 真机验证
图片输入	⚠️ 已实现但**未真机实测**（没往手机放图测试）
记忆抽取/展示	✅ 真机验证
记忆语义召回	⚠️ 每次对话已调用（无报错），但**语义效果未逐句确认**
端侧向量模型	✅ 仪器化测试通过（0.85 vs 0.39）

两件小事

1. **手机上留了测试数据**（1 轮对话 + 4 条记忆）。要清掉的话我可以执行 `adb shell pm clear com.isle.companion`，或你在应用里手动删——**你说了我再动**。
2. **图片输入**要不要我一并实测？我可以在你手机相册里不放入任何私人图片的前提下，用 adb 推一张测试图，再走一次「发图 → 角色回应图片内容」的流程。

需要我做哪个？

👤 用户 · 第 9 轮　10-15 16:39

> 记忆文案不要再用「用户……」这种第三人称冷叙述，改成角色第一人称、有温度的表述。

🤖 AI　10-15 16:39

明白，记忆要有人情味。我把「用户…」这种第三人称陈述，改成**角色自己心里的第一人称口吻**。

🤖 AI　10-15 16:39

构建通过。现在用**应用内的「清空记忆」功能**（而非删除数据）清掉旧措辞的记忆，再重新测一次。

🤖 AI　10-15 16:40

记忆已清空。现在回到聊天页发新消息，验证新的温暖措辞。

👤 用户 · 第 10 轮　10-15 16:40

> 请用中文完成一次端到端测试。

🤖 AI　10-15 16:41

先报个好消息：**记忆措辞已经改好了**，刚才那轮实测结果是：

> 记得七月二十号是你的生日，是夏天里海边最舒服的时候… 知道你周末喜欢去爬山，身上总带着一股很清爽的劲儿

再也没有「用户…」这种冷冰冰的表述了。

关于中文输入：`adb shell input text` 原生**不支持中文**（只能注入 ASCII），刚才英文被手机输入法自动纠错成了中文，所以看起来乱。我查一下你手机上可用的输入法，再决定怎么测。

🤖 AI　10-15 16:41

你手机上的三个输入法都不支持脚本注入中文（`input text` 本身只能发 ASCII）。要真正用中文测，只有两条路：

方案 1（推荐，我来做）：临时装一个开源调试输入法 **ADBKeyboard**（github 上的 dev 工具，专门用于 adb 注入中文） – 我会：装它 → 切为默认输入法 → 注入中文测试 → **切回你原来的输入法（腾讯 wetype）→ 卸载 ADBKeyboard** – 全程约 2 分钟，测完不留痕迹

方案 2（你自己来）：你在手机上手动打一句中文发出去，然后告诉我，我去看结果（不用装任何东西）

需要你点头：同意我装 ADBKeyboard 吗？（我装完会恢复原输入法并卸载它）

顺带说明：中文能力本身其实已经间接验证过了——角色回复和记忆抽取**本来就是中文**，只是"用户输入"这一步是英文转过来的。所以方案 1 主要是为了给你一个干净的中文端到端演示。

👤 用户 · 第 11 轮　10-15 16:41

> 中文测试已通过，继续。

👤 用户 · 第 12 轮　10-15 16:42

> 确保多角色记忆严格隔离；设置页增加「我的资料」（称呼、性格、自述），供所有角色共享。

🤖 AI　10-15 16:42

明白，两件事：**① 角色间记忆严格隔离；② 设置里加「我的资料」（名字/性格/自我介绍），让每个角色一上来就了解你**。

先确认现状：记忆表 MemoryEntity 带 characterId，所有查询（observeByCharacter / candidates / findByContent）都按角色过滤，隔离本来就是对的。我会再加一个测试来证明它。现在实现「我的资料」。

🤖 AI **10-15 16:43**

手滑打错了路径，重来。

🤖 AI **10-15 16:43**

加一个**记忆隔离测试**，用两个角色验证互不串记忆。

🤖 AI **10-15 16:43**

构建通过，但 8 秒不太可能跑完仪器化测试。核实测试是否真的执行了。

🤖 AI **10-15 16:43**

两个测试都通过了：memoriesAreIsolatedPerCharacter ✅ + embeddingProducesSemanticVectors ✅。安装并确认新入口渲染正常。

🤖 AI **10-15 16:44**

「我的资料」页面渲染正常。返回看一下设置主页的新入口。

🤖 AI **10-15 16:44**

完成，两点都落地了。

① 多角色记忆隔离（已用测试证明）

记忆全部挂在 characterId 下，所有读写都按角色过滤，且删除角色会级联清除其记忆。新增仪器化测试 MemoryIsolationTest：

```
创建「小屿」「阿泽」两个角色 → 各自写入记忆
→ 小屿只看到自己的记忆 ✅
→ 阿泽只看到自己的记忆 ✅
→ 删除小屿后其记忆清零，阿泽的记忆不受影响 ✅
```

测试结果：2 tests, 0 failures（隔离 + 向量推理）

② 设置新增「我的资料」（全局共享）

设计划分（正是你要的效果）：| 类型 | 范围 | 作用 ||---|---|---|| **我的资料** | 全局，所有角色共享 | 称呼、性格、想说的话 —— 新角色一见面就懂你 || **角色记忆** | 每个角色独立 | 你们之间的对话细节，绝不串台 |

设置页新入口：我的资料 · 填写后所有角色都能更快了解你，二级页三个字段： – **希望被怎么称呼**（例如：小彤） – **我的性格**（例如：慢热但很重感情，容易共情） – **想对 TA 说的话**（例如：我最近在准备项目，压力有点大）

内容会以这样的形式进入每次对话的系统提示词：

```
关于对方（你早就熟悉 TA 了，不需要再问）：
– TA 希望你称呼 TA 为「小彤」
```

- TA 的性格：慢热但很重感情
- TA 自己说过：最近在准备项目，压力有点大

改动文件

新增：data/prefs/UserProfileRepository.kt、ui/screen/settings/UserProfileViewModel.kt、UserProfileScreen.kt、androidTest/.../MemoryIsolationTest.kt **修改**：PromptBuilder.kt、AppContainer.kt、ChatViewModel.kt、SettingsScreen.kt、Routes.kt、IsleNavHost.kt

真机验证

- 设置主页已显示四个入口：我的资料 / 模型服务（已内置演示密钥 · deepseek-flash）/ 回复风格 / 关于心屿
- 「我的资料」页面三个输入框与说明文案渲染正常

需要我把「我的资料」也用在**记忆抽取**里吗？（现在抽取时没说"对方叫小彤"，模型只能从对话里听出来；加上后抽取会更准。）

👤 用户 · 第 13 轮　10-15 16:44

> 确认「我的资料」是否每轮都会发送给模型。

🤖 AI　10-15 16:44

是的，**每次对话都会发送**。具体链路：

位置	行为
ChatViewModel.kt:107	每轮发送前读取资料
ChatViewModel.kt:114 → PromptBuilder.kt:84	拼进**系统提示词**
结果	每轮请求都带着「角色卡 + 召回记忆 + 我的资料」一起发给模型

补充两点： – **不会**在「记忆抽取」时发送（MemoryExtractor 不含资料） – 其实不只资料——**整段系统提示词每轮都会重发**，这是对话类 API 的固有机制（无状态）。资料只占其中几十个 token，成本可忽略

⚠️ 但有个文案问题要提醒你

SettingsScreen.kt:147 现在写的是：

> 对话内容与 API Key 均仅保存在本机。

这句话**容易被误解**为"内容不出设备"。实际上：**本地只负责存储，但每轮对话内容（含资料）都会发送到 DeepSeek 云端 API 用于生成回复**。演示时如果评委较真，这会是个扣分点。

建议改成准确表述，例如：

> 对话记录与密钥仅保存在本机；发送消息时，内容会传输至你配置的模型服务用于生成回复。

`UserProfileScreen.kt:111` 的「资料仅保存在本机，会随每次对话提供给角色」也建议微调为「...会随每次对话发送给所配置的模型服务」。

需要你决定

1. **是否按上面修改这两处隐私文案？**（我建议改，更严谨也更可信）
2. 是否把资料也加进**记忆抽取**的提示词？（现在抽取时不知道"对方叫小彤"，知道后记忆会更准）

你回一下，我再动手。

👤 用户 · 第 14 轮　10-15 16:45

> 可以，按上述方案修改隐私文案，并把资料接入记忆抽取。

🤖 AI　10-15 16:46

编译无警告。安装并核对两处新文案是否生效。

🤖 AI　10-15 16:46

两处都改好并真机核对完毕。

① 隐私文案（已生效）

位置	新文案
设置页底部	对话记录与密钥仅保存在本机；**发送消息时，内容会传输至你配置的模型服务用于生成回复**。
我的资料页	提示：资料仅保存在本机，**会随每次对话发送给所配置的模型服务**，帮助 TA 更懂你。

② 记忆抽取接入「我的资料」

抽取时现在会带上已知信息，例如：

> 已知的对方信息：称呼「小彤」；性格：慢热但很重感情；自述：最近在准备项目

并在提示词里加了一条新规则：**「已知的信息不要重复记录，只记这次新出现的内容」**——避免每次聊天都生成"记得你叫小彤"这类重复记忆。

改动文件

- `ui/screen/settings/SettingsScreen.kt`（隐私文案）
- `ui/screen/settings/UserProfileScreen.kt`（隐私文案）
- `domain/memory/MemoryExtractor.kt`（新增 `userProfile` 参数 + `buildProfileHint()` + 去重规则）
- `ui/screen/chat/ChatViewModel.kt`（把资料传给抽取器）

验证

- `./gradlew :app:compileDebugKotlin --rerun-tasks` → **BUILD SUCCESSFUL，0 警告**
- 真机逐字核对：设置页、我的资料页文案均已更新 ✅
- ⚠️ 记忆抽取带资料这件事**没有做真机实测**（需要你在手机上填好资料后聊一句中文才能看出来）——你填完资料聊两句，我可以再去查记忆列表验证去重效果。

👤 用户 · 第 15 轮　10-15 16:47

> 既然已有记忆库，为什么资料不能只发送一次就长期生效？请解释其中的机制。

🤖 AI　10-15 16:47

你这个直觉很合理，但**大模型 API 是无状态的**——这就是必须每次重发的原因。

关键机制

`chat/completions` 这类接口，两次请求之间模型**什么都不记得**。它本质是一个函数：提示词进 → 文字出。

所谓的"记忆"，**并不存在于模型里**，而是存在**你手机的数据库里**。所以每轮对话前，我们必须把下面这些一起塞进提示词发过去：

```
角色卡（小屿是谁）
+ 我的资料（小彤是谁）
+ 检索出来的相关记忆
+ 最近 12 条对话
```

这不是我们的设计选择，是**所有聊天类大模型的通用做法**（包括 ChatGPT、Claude）。所谓 RAG / 知识库，本质就是"每次检索出相关片段，再喂给模型"。

一个类比

模型像一个**绝顶聪明、但每次说话都会失忆**的人。 你每次开口前，都得先递给 TA 一张小纸条：「这是小彤，她慢热但重感情，上次她说在准备项目」。

递纸条这件事看起来"重复"，但**正是它让 TA 表现得像记得你**。

那"发一次就永久记得"能做到吗?

能，但那是另一种架构： | 方案 | 谁保存状态 | 说明 | |---|---|---| | 现在我们这样 | 你的手机 | 每轮重发上下文（标准做法） | | 服务端会话/助手 API | 模型厂商 | 由厂商托管上下文，但仍会在后台重新喂给模型 | | 微调模型 | 厂商 | 把知识"烧"进权重，成本高、更新慢，不适合个人资料 |

DeepSeek 提供的是标准 `chat/completions`，没有会话托管；而且**个人资料随时会改**，塞进权重也不现实。

成本其实不用担心

- 资料那段只有几十个 token，一轮请求里系统提示词整体也就几百 token

- DeepSeek 有**上下文前缀缓存**机制，重复的 system 前缀通常会被缓存命中，费用进一步降低（具体以官方计费文档为准）

所以：**"每轮都发" ≠ 浪费，而是"每次让 TA 想起你"的唯一方式**。真正省 token 的优化方向是——把长历史压成摘要，只留最相关的记忆（我们已经在这么做了）。

👤 用户 · 第 16 轮　10-15 16:48

> 「关于心屿」保留在设置页内，其余入口改为二级页面。

🤖 AI　10-15 16:48

这句我有两种理解，怕改错，先确认一下（选一个就行）：

理解 A（我的倾向）：设置页做成**纯入口列表**——把现在还在设置页内的「回复风格」也改成二级页面，于是四项统一为： 我的资料 →、模型服务 →、回复风格 →、关于心屿 →

理解 B：把「关于心屿」的内容（项目介绍/背景/开发者）**直接展开在设置页里**，不再单开一页；其余三项保持二级页面。

你要 A 还是 B?

👤 用户 · 第 17 轮　10-15 16:49

> 采用方案 B。

👤 用户 · 第 18 轮　10-15 16:49

> 确认方案 B。

👤 用户 · 第 19 轮　10-15 16:50

> 已理解，按 B 执行。

🤖 AI　10-15 16:50

明白，B。开始改：设置页内直接展开「关于心屿」全文；我的资料 / 模型服务 / 回复风格 三项统一走二级页面。

🤖 AI　10-15 16:50

导入正确。删除已废弃的 `AboutScreen.kt`（内容已移入设置页），然后构建。

🤖 AI　10-15 16:50

编译干净。安装后实际看效果。

🤖 AI　10-15 16:51

手机又锁屏了。唤醒后重试。

👤 用户 · 第 20 轮　10-15 16:51

> 把三个功能入口放到项目介绍下方，顺序更符合阅读习惯。

🤖 AI　10-15 16:51

明白，把「关于心屿」内容放最上面，三个入口移到简介下面。

🤖 AI　10-15 16:52

顺序已调好——关于内容在最上。滚动到底确认三个入口在下方。

👤 用户 · 第 21 轮　10-15 16:52

> 将「技术亮点与开发者」再拆为一个独立二级页面。

🤖 AI　10-15 16:52

明白，把「技术亮点」和「开发者」单独拆到二级页面。

🤖 AI　10-15 16:53

技术亮点二级页渲染正确。返回确认设置页四个入口。

👤 用户 · 第 22 轮　10-15 16:54

> 清理无用代码与依赖。

🤖 AI　10-15 16:54

我来系统扫一遍无用文件与死代码。

🤖 AI　10-15 16:54

别名扫描方式有误（构建文件用 `libs.xxx.yyy` 点号形式）。我逐个核实可疑符号的真实使用情况。

🤖 AI　10-15 16:55

扫描结果清晰。开始清理（每项都已确认 0 引用）：

本幕小结

- **设置体系**：关于内嵌 + 我的资料 / 模型服务 / 回复风格 / 技术亮点四个二级入口；
- **密钥方案**：`local.properties` 构建时注入 `BuildConfig`，用户 Key 优先于演示 Key；
- **隐私口径**：从「仅保存在本机」修正为「本地存储，发送消息时传输至模型服务」；
- **记忆温度**：抽取提示词改为第一人称，禁止出现「用户」二字；
- **可验证性**：新增 `UserProfile` 与记忆隔离测试，把「不可见」的约束写成断言。

第三幕 · 体验缺陷攻坚（10–15 16:55–17:18）

> **本幕目标** 用证据链定位三个高频问题：图片模糊、输入法、UI 质感。
> **验收标准** 每个问题都能说清「是 App 还是环境」「根因在哪一层」「修复后如何验证」。

👤 用户 · 第 23 轮　10–15 16:55

> 严重问题：聊天页点击输入框无法唤起输入法，请定位根因。

🤖 AI　10–15 16:55

先放下清理，优先查这个 bug。我上真机诊断输入法状态。

🤖 AI　10–15 16:55

默认输入法是 **`com.tencent.wetype/.plugin.hld.WxHldService`**——这是微信输入法的一个「插件服务」，不是真正的主键盘。这很可能就是原因。我先做决定性诊断。

🤖 AI　10–15 16:56

键盘立刻出现了 —— 决定性证据：App 侧完全没有问题。

现在把你的手机留在可用状态（Google 键盘），同时我做一处 App 侧的健壮性加固（键盘弹出时输入框会被顶到键盘上方）。

🤖 AI　10–15 16:57

键盘正常弹出，且**输入框被顶到键盘上方**（布局正确让位）。

不过我发现刚才的清理只做了代码部分，**依赖还没清**。补上。

👤 用户 · 第 24 轮　10–15 16:58

> 微信输入法仍然无法启动。

🤖 AI　10–15 16:58

那就说得通了 —— **你的微信输入法本身在这个手机上起不来**，而我诊断到的默认输入法恰好是它的一个「插件服务」（`plugin.hld.WxHldService`），所以任何 App 都拉不出键盘。

我先装好清理后的新包并做冒烟验证。

🤖 AI　10–15 16:58

冒烟验证通过（无崩溃），但同时暴露了一个关键情况：**微信输入法把默认输入法又抢回去了**——我切到 Google 键盘后，它自己又设回了那个坏掉的插件服务。这就是你"启动不了"的根本原因：它一直在抢占默认位置，却渲染不出键盘。

本次完成的事

① 查清「键盘拉不起来」= 设备问题，不是 App – 证据：`mInputShown=true`、`mServedView=AndroidComposeView`、IME 窗口存在、输入框有焦点 —— App 请求与连接全部正常；换 Google 键盘后**立刻出键盘**

② App 侧加固（顺带修掉一个真问题） – `AndroidManifest.xml`：`android:windowSoftInputMode="adjustResize"` – `ChatScreen` + 4 个表单页：加 `imePadding()`，键盘弹出时输入框自动上移，不会被挡住

③ 无用代码与依赖清理（构建已验证） | 类型 | 清理项 | |---|---| | 死代码 | `EmbeddingProvider.NoOp`、`ContentFactory.emptyObject()`、`SettingsRepository.updateMaxTokens`、`AvatarStorage.file()`、`ChatRepository.observe()/clear()`、`MessageDao.observeByCharacter()/clearByCharacter()` | | 依赖 | `okhttp-logging`（未用）、`ui-tooling-preview`（无 @Preview）、`room-ktx`（Room 2.6+ 已并入 runtime） | | 版本目录 | 未使用的 `kotlin-android` 插件别名 |

验证：依赖删除后全量重建 **BUILD SUCCESSFUL（42s）**，安装运行无崩溃。

需要你决定

你的手机输入法要我怎么处理?

1. **禁用坏掉的微信输入法服务 + 设 Google 键盘为默认**（我能用 adb 一条命令搞定，之后打字就正常了）
2. **你自己在手机「设置 → 系统设置 → 键盘与输入法」里关掉微信输入法**（更稳妥，我不动你手机）
3. 只把默认切回 Google 键盘（但微信输入法可能又会抢回去，治标不治本）

我建议 **2**（你自己操作最干净），或者授权我执行 **1**。

👤 用户 · 第 25 轮　10-15 16:59

> 已恢复正常。

👤 用户 · 第 26 轮　10-15 17:00

> 图片消息测试通过。

🤖 AI　10-15 17:00

好，文档先放着。

那剩下一项：**要不要我打一个 release 包**（开 R8 混淆 + 资源压缩，体积能从 67MB 降下来）？

签名需要你选一种：

1. **生成正式 keystore**（推荐用于提交）—— 我生成密钥库，密码写进 `local.properties`（不入库），你自行保管；以后升级必须用同一个 keystore
2. **先用 debug 签名**（最快，能装能演示，但不适合正式上架/长期迭代）
3. **你自己已有 keystore** —— 把路径、别名、密码告诉我，我配置进去

选哪个？或者暂时不打包也行。

用户 · 第 27 轮　10-15 17:00

> 交付材料和 release 包暂时不需要。

本幕小结

问题	根因	修复
图片点开模糊	聊天图复用了头像 512px 压缩	抽出 `maxSize`，聊天图 1600px；查看器改 `fillMaxSize`
输入法唤不起	设备默认输入法指向微信输入法插件服务	`dumpsys input_method` 判定 App 请求正常；App 侧补 `adjustResize` + `imePadding`
图片消息验证	多模态链路已实现	真机发送截图，AI 准确描述页面内容

第四幕 · 液态玻璃与视觉重构（10-15 17:18—17:42）

> **本幕目标** 从「功能可用」走到「视觉统一」：设计规范、真实背景模糊、包体优化。
> **验收标准** 9 个页面风格一致；顶栏/输入栏有真实 backdrop blur；APK 按 ABI 拆分。

用户 · 第 28 轮　10-15 17:18

> 聊天页右上角改为「编辑角色」；图片要支持全屏查看；整体 UI 需要提升质感。

AI　10-15 17:19

手滑留了段死代码，清掉。

用户 · 第 29 轮　10-15 17:22

> 统一为液态玻璃风格，并修复图片查看模糊的问题。

AI　10-15 17:22

两个问题都处理：① **图片糊是因为聊天图片被压缩到了 512px**（头像压缩逻辑被复用了）；② **全面改为液态玻璃质感**。先修图片。

AI　10-15 17:22

现在做液态玻璃主题。

AI　10-15 17:22

zsh 不做变量分词导致循环失败，改用 bash 执行。

🤖 AI　10-15 17:23

我用的去重命令误删了文件里的重复行（awk 全局去重把合法重复行也删了）。重写该文件。

🤖 AI　10-15 17:24

构建通过。最后统一顶栏的玻璃质感。

👤 用户 · 第 30 轮　10-15 17:25

> 右下角悬浮加号不美观；首页移除编辑角色入口。

🤖 AI　10-15 17:25

两处都改。

🤖 AI　10-15 17:25

IconButton 在顶栏设置按钮还在用，把导入加回来。

👤 用户 · 第 31 轮　10-15 17:26

> 视觉仍不理想，且全局 UI 不统一，请系统性排查。

🤖 AI　10-15 17:26

明白，我做一次**系统性的设计统一**，而不是零散修补。核心问题我先说：**主题开了 Material You 动态取色**，导致系统色和我们的品牌紫打架——这是"不统一"的根源。

🤖 AI　10-15 17:27

现在重做首页布局（去掉悬浮按钮，改为列表内的「创建新角色」卡片，并统一间距与圆角）。

🤖 AI　10-15 17:27

统一各页圆角与输入框形态。

👤 用户 · 第 32 轮　10-15 17:29

> 目标视觉是 iOS 那种液态玻璃：通透、磨砂、有真实背景模糊。

🤖 AI　10-15 17:29

这就是真正的「背景模糊」玻璃了——我现在的实现只是半透明白色，不是真玻璃。要达到 iOS 那种"透透的"效果，需要**真实背景模糊（backdrop blur）**。我先确认方案可行性。

🤖 AI　10-15 17:29

全部 2.0.1 可用。我查一下具体 API 用法（避免反复试错）。

AI 10-15 17:30

直接用编译器验证 API。先加依赖。

AI 10-15 17:31

Haze 2.x 用 Kotlin 2.4 编译，与我们 2.2.0 编译器不兼容。检查 1.7.x 是否可用。

AI 10-15 17:31

haze 1.7.3 依赖 stdlib 2.3.20（编译器可读）。切到 1.7.3。

用户 · 第 33 轮 10-15 17:32

> 是否可以升级 Kotlin，以解锁新版 Haze?

AI 10-15 17:32

可以，但有个前提要先确认。我先查清升级路径上的关键依赖（KSP 是否支持 Kotlin 2.4）。

AI 10-15 17:32

Kotlin **2.3.21 + KSP 2.3.12** 都齐。升级方案：关闭 AGP 内置 Kotlin，改用独立 Kotlin 插件 2.3.21（编译器可读 2.4.0 元数据 → Haze 2.x 解锁）。

AI 10-15 17:33

官方给出了明确绕过方式：`android.newDsl=false`。

AI 10-15 17:33

Kotlin 2.3.21 已生效（元数据错误消失）。只剩 Haze 2 的 API 包名要对。我直接查 jar 里的类。

AI 10-15 17:34

`CupertinoMaterials`（Apple 风格材质）正好可用！再确认 `HazeInput` 与 `hazeBlur` 签名。

AI 10-15 17:34

API 已摸清。写最终版本。

AI 10-15 17:35

包名搞错了（我查的是 1.7.3 的 jar）。查 2.0.1 的 materials 类。

AI 10-15 17:35

Haze 2.0.1 编译通过！现在应用到界面。先改首页。

AI 10-15 17:35

首页已用上真玻璃。现在改聊天页（消息从顶栏下方滚过 → 真实模糊）。

AI 10-15 17:35

聊天页完成。批量应用到其余页面（设置/记忆/表单页）。

🤖 AI　10-15 17:35

7 个页面全部接入。再把玻璃调得更"透"，然后实机验证。

👤 用户 · 第 34 轮　10-15 17:37

> 磨砂质感不足，请继续增强。

🤖 AI　10-15 17:37

"更透 + 更磨砂"——我用自定义 Haze 样式（大模糊半径 + 低不透明度 + 噪点磨砂纹理）。先看样式作用域支持哪些参数。

🤖 AI　10-15 17:37

作用域支持 `blurRadius` / `noiseFactor`（磨砂颗粒）/ `backgroundColor`。写自定义样式。

🤖 AI　10-15 17:38

磨砂感出来了——上图顶栏区域能看到英雄卡片滚动到下方后，被**明显模糊成一片紫蓝色雾气**透出来，这就是 iOS 液态玻璃的核心效果。

本轮调整（更透 + 更磨砂）

参数	原值	现值	作用
`blurRadius`	材质默认	**48dp**	强磨砂，背后内容糊成色块
`noiseFactor`	默认	**0.14**	玻璃表面颗粒感（磨砂的关键）
玻璃底色 alpha	0.16	**0.10**	更透，颜色从下面透上来
卡片填充 alpha	0.58→0.34	**0.40→0.18**	卡片也明显透光
卡片描边	0.9	0.75	高光更柔和自然

你应该能明显感觉到的两处

1. **首页**：卡片现在能透出后面的渐变底色，不再是"实心白块"
2. **滚动时**：内容滚到顶栏下方，会**实时被模糊成雾气**——这是之前完全做不到的（之前只是半透明白，没有模糊）

再看看效果，如果还想调： – 要**更糊**→ 我加 `blurRadius` 到 64+ – 要**更透**→ 继续降 alpha – 要**颗粒更重**→ 提高 `noiseFactor`

👤 用户 · 第 35 轮　10-15 17:39

> 开发者信息改为三行：沧州交通学院 / 计算机与信息技术学院 / 软件工程 2410 吴思彤。

🤖 AI　10-15 17:40

已改为三行：沧州交通学院 / 计算机与信息技术学院 / 软件工程 2410 吴思彤。构建安装后用 uiautomator 核对，真机显示正确。

👤 用户 · 第 36 轮　10-15 17:41

> 首页顶部模糊过重，请收敛参数。

🤖 AI　10-15 17:42

把 `blurRadius` 从 48dp 收敛到 22dp、`noiseFactor` 0.14→0.06、玻璃底色 alpha 0.10→0.14；重新安装截图，首页顶部不再糊成色雾。

👤 用户 · 第 37 轮　10-15 17:42

> 阶段性收尾，当前版本先到这里。

🤖 AI　10-15 17:42

本阶段收尾：核心功能、视觉体系与真机验证已完成；release 打包、交付材料与测试数据清理列为后续项。

本幕小结

项	结果
视觉统一	关闭 Material You 动态取色；`DesignSystem.kt` 固化圆角/间距
液态玻璃	Haze 2.0.1；`blurRadius 22dp`、`noiseFactor 0.06`、底色 alpha 0.14
工具链	Kotlin 2.3.21 + KSP 2.3.12；`builtInKotlin=false` + `newDsl=false`
交互	去掉悬浮 FAB，改为列表内「＋ 创建新角色」；聊天页右上角改为编辑角色
包体	185MB → 67MB（arm64）/ 58MB（armv7）

第五幕 · 从代码到交付材料（10–16）

> **本幕说明** 核心功能完成后，项目进入「可交付」阶段：文档、截图、PDF、仓库与演示素材。以下为要点还原。

👤 用户 · 10–16 08:05

> 请按交付要求梳理完整开发流程；只写真实发生的过程，不要为了好看而虚构。

AI · 10–16 08:07

先对齐交付清单：公开仓库、细粒度提交、README、5–8 分钟演示视频、不超过 30 页的技术文档，以及 AI 对话记录。随后按真实开发顺序整理材料，不编造未发生的步骤。

用户 · 10–16 08:07

> 把能产出的材料全部产出。

AI · 10–16 08:09

产出 README、开发全流程记录、技术文档三份基础材料，并把 10–15 的开发成果按模块整理为 12 个基础提交；随后补充 AI 协同记录与对话导出。

用户 · 10–16 08:23

> 请从会话数据库导出 AI 对话历史，并对密钥、令牌等敏感信息脱敏。

AI · 10–16 08:31

从 `opencode.db` 只读导出会话，按 `ghp_` / `github_pat_` / `sk-` 规则脱敏；导出文件随后按「只保留立项后内容」的原则多轮收敛。

用户 · 10–16 08:43

> 技术文档调整为适合 PDF 的版式：封面、图表位置、分页建议，并直接导出 PDF。

AI · 10–16 08:45

用 Markdown → HTML → Chrome Headless 生成 PDF；补封面元数据、配图占位与打印样式，首版 16 页。

用户 · 10–16 08:51

> 需要配图的地方用真机截图，手机已连接。

AI · 10–16 09:16

通过 adb 抓取首页、聊天、记忆、角色编辑、设置、我的资料等真机截图，替换正文占位；后续补齐终端验证图与架构图，最终 24 页。

用户 · 10–16 11:10

> 再写一份用户使用说明书，版式与技术文档一致，配真机截图并导出 PDF。

AI · 10–16 11:14

产出 17 页说明书，覆盖年龄确认、安装、角色、聊天、资料、记忆、设置、隐私与常见问题。

用户 · 10–16 11:30

> 为录屏设计一个「AI 定位编译错误」的复现 Prompt，要求先查证据再给方案。

AI · 10–16 11:38

给出「报错原文 + 依赖树 + 环境事实 + 已尝试修复」的证据链模板；真机复现 Kotlin metadata 2.4.0 报错，定位到 KSP 版本写错并恢复为 2.3.12。

本幕小结

交付物	结果
README	10 分钟复现指南：环境、构建、安装、测试、排错
技术文档	24 页，含意图控制、架构、端侧记忆、提示词、可复现性
使用说明书	17 页，含 9 张真机截图
开发日志	五阶段产出 + 13 个真实问题
AI 记录	协同 Prompt 链 + 对话节选
仓库	Public；基础提交 12 个，后续按真实时间继续提交

第六幕 · 未成年保护与最终交付（10–17）

> **本幕说明** 这是产品从「功能完成」到「合规可交付」的关键转折：仅靠弹窗告知不够，必须做分级限制。

用户 · 10–16 22:30

> 考虑到未成年人保护：选择「恋人」「自定义」关系时弹出免责声明，并在设置页补充同样的说明。

AI · 10–16 22:31

实现「选择亲密关系前二次确认」弹窗，并在技术信息页增加未成年人保护提示。但随后发现一个绕过路径。

用户 · 10–16 22:33

> 新建角色的默认关系是「恋人」，用户不点选项直接保存就能绕过弹窗；默认值改成中性类型。

AI · 10–16 22:34

默认关系从 `love` 改为 `bestie`；只有主动选择「恋人/自定义」才触发声明。验证通过并提交。

用户 · 10–17 14:19

> 仅靠弹窗声明仍然无法阻止未成年人使用；请给出更有效的分级方案。

AI · 10–17 14:20

承认「弹窗只是告知，不是拦截」，给出四层方案：年龄门、青少年模式、防沉迷提醒、合规文案；并说明本地应用无法做实名核验，自声明是能力边界。

用户 · 10–17 14:21

> 实现第 1、2 层（年龄门 + 青少年模式），并移除之前点击「恋人」时的弹窗。

AI · 10–17 14:22

新增首次启动年龄门（未确认不进入主界面）；青少年模式下隐藏「恋人」「自定义」，系统提示词追加最高优先级保护约束，记忆抽取禁止亲密内容；设置页显示「青少年模式」。原弹窗与对应截图一并移除。

用户 · 10–17 14:25

> 补充年龄门与青少年模式的真机截图，并更新文档与 PDF。

AI · 10–17 14:28

抓取年龄门、青少年模式关系类型两张截图；技术文档新增「未成年人保护设计」章节，说明书新增年龄确认章节；PDF 重新生成（技术文档 24 页、说明书 17 页）。

用户 · 10–17 14:33

> 说明书的年龄确认章节应该放在首页；调整章节顺序。

AI · 10–17 14:34

把「年龄确认与青少年模式」提为第一章，全书顺延为 12 章；删除已废弃的弹窗截图，最终 17 张图全部被引用。

本幕小结

机制	落地
年龄门	首次启动确认年龄段，未确认不进入主界面
青少年模式	隐藏「恋人」「自定义」；设置页标识

机制	落地
内容约束	系统提示词最高优先级；禁止亲密称呼与恋爱角色扮演
记忆约束	不记录涉及亲密关系的内容
能力边界	年龄为自声明；实名核验需服务端与合规资质

第七幕 · 交付后修复（10–17 15:00–15:30）

> **说明** 以下为交付验收后新增的修复，不在原始 37 轮对话内，摘自开发日志与提交记录。修复后重新全量编译，并在真机执行仪器化测试。

#	验收发现	修复
1	冷启动时年龄门闪一下	区分「配置未加载 / 未确认」，加载完成前只渲染背景
2	编辑角色后列表顺序变化	更新时保留原 `createdAt`
3	同毫秒消息排序可能错位	`ORDER BY timestamp DESC, id DESC`；助手 id 取插入返回值
4	SSE 完成回调可能重复结算	`AtomicBoolean` 保证只结算一次
5	ONNX 输入张量未释放	`OnnxTensor.use { }` 显式关闭
6	「仅存本机」与系统云备份矛盾	`allowBackup=false`，不参与云备份/换机迁移
7	聊天页附图弹窗文案错误	图片选择器支持标题，聊天页显示「发送图片」
8	记忆解析只看首尾方括号	实现 `removeThinkTags()` + `extractJSON()`

验证

```
./gradlew compileDebugKotlin --offline --rerun-tasks
BUILD SUCCESSFUL

./gradlew connectedDebugAndroidTest
2 tests, 0 failures
```

真机复测：年龄门、编辑角色顺序、消息顺序、图片弹窗、联网流式回复、记忆抽取与召回（2 条记忆、512 维向量）均通过。

成品状态

维度	状态
应用	心屿 v1.0.0，原生 Android，`minSdk 26` / `compileSdk 37`

维度	状态
角色	预置小屿 / 阿岸 / 林深 + 多角色自定义（性格/背景/说话风格/头像）
对话	DeepSeek SSE 流式；支持图片多模态；可停止、失败重试、清空会话
记忆	LLM 抽取 + 端侧 512 维向量 + 本地加权召回 Top-6
隐私	数据本地存储；不参与云备份；仅发送时传输至模型服务
合规	年龄门 + 青少年模式 + 提示词/记忆双重约束
演示数据	3 个角色 / 6 条记忆 / 7 条示例对话，可重置
质量	Kotlin 0 警告；2 tests / 0 failures；arm64 APK 68MB
交付	6 份 Markdown 文档 + 2 份 PDF + 17 张截图 + 40+ 次提交

后续迭代 · 交付前收口（10-26～10-29）

> **说明** 以下为交付后新增的收口工作，不在原始 37 轮对话内，按需求变更与提交记录摘要整理。

需求变化：按交付要求收口为**单版本**，保留主包名 `com.isle.companion` 与 `v1.0.0`。

本轮改动

方向	内容
演示数据	3 个角色、6 条记忆、7 条示例对话；设置页可重置
聊天体验	新增停止生成、失败重试、清空会话；错误不写入历史
图片	采样解码、UUID 文件名、删角色/清空会话清理图片、拍照缓存清理
设置	年龄分组重选、最大 tokens、测试连接/余额、演示数据重置
记忆	语义近似去重（余弦 ⩾ 0.92）
数据	Room v2 + 复合索引 + Migration(1→2)
视觉	窗口背景对齐主题，新增深色 `values-night`

验证：`assembleDebug` 通过；演示数据 3/6/7；测试连接返回余额；重置与年龄重选可用；覆盖安装验证过 v1→v2 迁移。

收口结果：单版本，`com.isle.companion`，v1.0.0，arm64 68MB。

后续迭代 · 模型服务通用化（10-29）

> **说明** 以下为交付后新增的模型接入改造摘要，不在原始 37 轮对话内。

方向：从 DeepSeek 单厂商改为 OpenAI 兼容协议，地址 / 模型 / Key 全部可配置，模型可在设置页切换。

方向	内容
协议	`OpenAiClient` + `OpenAiUrls` + SSE / 错误解析，任意 `/chat/completions` 服务
配置	`AI_BASE_URL` / `AI_MODEL` / `AI_API_KEY` 构建期注入，应用内设置优先
切换	OpenAI / DeepSeek / 通义千问 / 千问 MaaS / 本地 Ollama 预设；`/models` 可搜索列表点选
本地	支持 `http://` 本地 / 局域网服务，Key 可留空
模拟器	ONNX 指令不兼容时自动降级关键词匹配

验证：33 个 JVM 单测通过；3 个真实接口 E2E 全过；千问 MaaS `qwen3.5-omni-plus` 的 `/models` 返回 184 个模型，非流式与流式回复正常。

> 从一句「可以重构为原生安卓吗」，到一款能记住你、能通过真机验收的应用—— 这份记录保留的不是「AI 一次写对」的神话，而是**每次报错、每次定位、每次修正**的真实轨迹。
